

Deliverable 2

Modelling, Model Selection & Evaluation Report

Dataset B: PlantVillage — Plant-Disease Classification

Plant Recognition · Capstone Project · Step 3

Eleven models were built to recognise crop diseases from photographs of single leaves, compared on one shared benchmark, and reduced to one. This report sets out the comparison, the choice, the evaluation of that choice against a partition of the data kept sealed throughout development, and what can be seen inside the model when it makes a decision. Part II records, model by model, the technical detail behind every number.

Cohort may26bds_int

Prepared by the project team

Adrian Bogdan RUS · Alexander DANZ · Christoph Clemens Martin NEDDENS · Scheima Sara OBEIDI

Project mentor: Habiba El Hussein (ML Engineer)

30 July 2026

Contents

PART I — MODELLING REPORT

	Executive summary	4
1	Introduction	4
2	The problem, and the measure of success	5
3	Making eleven models comparable	5
4	The ladder at a glance	6
5	What the classical models taught us	7
6	What the convolutional network taught us	8
7	What transfer learning taught us	8
8	Choosing the final model	9
9	The final model, and the sealed test set	10
10	Where the remaining errors are	11
11	Looking inside the model	11
12	Limitations	13
13	Outlook and conclusion	13

PART II — TECHNICAL APPENDIX

A	Common ground — what every model shared	16
A.1	Compute environment	16
A.2	The canonical split	16
A.3	The image pipeline	16
A.4	Class weights	17
A.5	Augmentation	17
A.6	How the metrics were computed	17
A.7	Reproducibility conventions	17
B	Model records	20
B.1	Dummy — most frequent	20
B.2	Dummy — stratified	20
B.3	Logistic regression on six global features	21
B.4	Random Forest on six global features	22
B.5	Random Forest on ~160 features — the baseline	23
B.6	The scratch CNN	28
B.7	The tuned CNN	34
B.8	Transfer — MobileNetV2	35
B.9	Transfer — ResNet50	37
B.10	Transfer — EfficientNetB0	37
B.11	DenseNet-121 — the final model	38
B.11.1	Why this backbone, among the four	38
B.11.2	Dense connectivity	39
B.11.3	Inside a dense block	40
B.11.4	The network at 128 pixels	41
B.11.5	The model as assembled	42
B.11.6	The parameter budget	43
B.11.7	The two-phase protocol	43
B.11.8	Why the normalisation layers stay frozen	44
B.11.9	The assertion that fine-tuning engaged	45
B.11.10	Callbacks, checkpointing and what was saved	45
B.11.11	Compute	45
B.11.12	Validation and the sealed test set	46
B.11.13	How the sealed evaluation was enforced	46
B.11.14	Where the errors are	47
B.11.15	Grad-CAM — mechanics, and a bug worth recording	49
B.11.16	Turning heat maps into a number	49
B.11.17	The grid-resolution confound	51
B.11.18	Attention as a runtime confidence signal	51
B.11.19	What it costs to serve	51

	B.11.20	Reproducing this model	52
	B.11.21	The selection record	52
C		Cross-model ledger	54

Modelling Report

Executive summary

This report describes how eleven models were built to recognise crop diseases from photographs of single leaves, how they were compared, which one was chosen, and how well it performs. The chosen model is DenseNet-121, a network pretrained on ImageNet and fine-tuned on our data. On the sealed test set it reaches a macro-F1 of 0.9919 with a 95% confidence interval of [0.9896, 0.9941], an accuracy of 0.9930, and 57 errors in 8,146 images. Its validation score was 0.9918, so the difference between the score used to develop it and the score on data it had never seen is +0.0001. There is no measurable over-fitting.

The route to that model was a ladder rather than a search. Each rung was built to answer a question the rung below had raised. Two no-skill baselines fixed the floor and showed why accuracy is the wrong measure on a dataset where the largest class outnumbers the smallest thirty-six to one. A pair of classical models on the same six features separated what the choice of algorithm is worth from what the choice of features is worth, and the answer was decisive: the features matter roughly five times more. A richer classical model reached 0.906 and marked the ceiling of hand-designed descriptors. Convolutional networks then cleared that ceiling by learning where a lesion sits rather than only what colour it is, and a pretrained backbone cleared the from-scratch network.

The decision to take DenseNet-121 forward rests on validation performance, where it leads the from-scratch network by 0.9918 to 0.973. That margin is close to the run-to-run variation this project has documented, so Section 8 sets out what supports it. The strongest argument is forward-looking: DenseNet-121 is much the better foundation for the version of this tool that will eventually have to work in a field. Pretrained weights pay off most when training data is scarce; our laboratory dataset is not scarce, but field imagery will be, and adapting a backbone that already carries general visual structure to a few thousand in-situ photographs is a shallow re-fine-tune rather than a fresh start. Alongside that, the model removes errors where they are hardest to remove — the worst-recalled class sits at 0.948, only one class of thirty-eight falls below 0.95, and twenty are recalled perfectly.

Grad-CAM was used to see where the model looks when it makes a decision. It confirms that the model attends preferentially to the leaf, and it produces something practically useful: on the images the model gets wrong, its attention is spread as though the leaf were not there, which makes attention concentration a cheap way to flag doubtful predictions for human review. It also shows something worth stating plainly — the pretrained model does not attend to the leaf any more than the from-scratch network does, so the report makes no claim that it will survive field conditions unchanged.

Every number in this report is a laboratory number. PlantVillage is detached leaves photographed flat against a plain backdrop, and neither model has ever seen anything else. What the work delivers is a well-understood model, a documented comparison behind it, and a clear view of what would have to happen next.

1. Introduction

Deliverable 1 described the dataset and what could be seen in it. PlantVillage holds 54,305 photographs of individual leaves, sorted into 38 classes that combine a crop with a condition — fourteen crops and twenty-one distinct conditions, including healthy tissue. Every image is a single leaf, detached from the plant and photographed flat against a plain, evenly lit backdrop. The exploration in Deliverable 1 reached two conclusions that shaped everything in this report: the classes are separated by fine local detail rather than by overall colour, and the dataset is badly imbalanced.

This report covers what was built on top of that. Eleven models were trained, compared on a single shared benchmark, and reduced to one. That one model was then evaluated once against a partition of the data that had been kept sealed throughout development. The report explains why each model was built, what each one revealed, why the final choice was made, where the remaining errors are, and what can be seen inside the model when it makes a decision.

It is written to be read straight through. Sections 2 and 3 set up the measure of success and the conditions under which the models were compared, without which none of the numbers mean anything. Sections 4 to 7 walk up the ladder of models and draw out what each tier taught. Sections 8 to 10 cover the choice of final model, its verification against the sealed data, and its errors. Sections 11 to 13 look inside the model, set out what the work does not establish, and say what would come next.

One note on naming. Three models are referred to often enough to deserve short names. The Random Forest built on roughly 160 classical features is **the baseline**— the strongest result achievable without deep learning, and the mark the neural networks had to beat. The convolutional network trained from random initialisation is **the scratch CNN**. DenseNet-121 is **the final model**.

2. The problem, and the measure of success

What kind of problem this is

The task is supervised, single-label, multi-class classification. The input is a 128×128 colour photograph of one leaf; the output is exactly one of 38 mutually exclusive labels. There is no ordering among the labels and no continuous quantity to predict, so this is not a regression problem, and because every training image arrives with a ground-truth label supplied by the dataset's own folder structure, it is not a clustering problem either.

Within image recognition it belongs to the harder end: fine-grained classification. The distinction the model has to draw is not between a cat and a car but between Tomato Early blight and Tomato Late blight, two conditions that affect the same crop and produce lesions on tissue that is otherwise identical. The discriminating signal is often a few square centimetres of texture. The intended application is a diagnostic aid — a grower photographs a leaf and receives a likely disease — which shapes both the metric and the way errors are judged later in this report.

It is worth being concrete about what the 38 labels contain, because the structure of the label set explains much of what appears in the error analysis later. Tomato alone accounts for ten classes, Apple and Corn for four each, and several crops — Blueberry, Raspberry, Soybean — appear only as healthy. Twelve of the thirty-eight classes are healthy tissue. This means the model is asked to make two quite different kinds of distinction at once: between a diseased leaf and a healthy one, which is usually easy, and between two diseases of the same crop, which is often hard. The classes are not equally difficult, and an average across them will always conceal that.

Why macro-F1 is the measure

All comparisons in this report are made on macro-F1: the unweighted average of the per-class F1 scores, so that each of the 38 classes contributes equally regardless of how many images it has. The alternative, plain accuracy, weights every image equally, which on this dataset means weighting the common classes heavily and the rare ones barely at all. The imbalance is severe — the largest class, Orange Huanglongbing, has 5,507 images; the smallest, healthy Potato, has 152.

The two dummy models in Table 1 turn that from an argument into a demonstration. A model that does nothing but predict the largest class every time achieves 10.1% accuracy. That is a respectable-looking number, and it describes a model with no ability whatsoever. Its macro-F1 is 0.005. For a diagnostic tool the rare disease is precisely the one a grower most needs identified, so the measure has to be one that refuses to average it away.

What else is reported

Macro-F1 is the decision metric, but a single number cannot say much about how a model fails, so three other views are used throughout.

- **Accuracy** appears alongside macro-F1 for continuity with the published work on this dataset, and because the gap between the two is informative in its own right — a wide gap means the model is doing well on the common classes and poorly on the rare ones.
- **Per-class recall** is what an average conceals. It names the disease being missed rather than reporting that some disease is being missed. The lowest per-class recall is quoted for the final model as a worst-case guarantee.
- **The confusion matrix** records not only how often the model is wrong but what it mistakes for what. That distinction matters: confusing two diseases of the same crop is a different kind of failure from confusing a diseased leaf with a healthy one, and Section 10 rests on it.

3. Making eleven models comparable

Eleven numbers only form a comparison if they were produced under the same conditions. Four things were fixed before any model was trained, and none of them changed afterwards.

- **One data split, shared by everyone.** All 54,305 images were divided once, stratified by class, into 70% training (38,013 images), 15% validation (8,146) and 15% test (8,146), with a fixed random seed. The split was distributed to

the team as CSV files rather than regenerated locally, and every notebook prints the split's fingerprint at startup and halts if it does not match. That guard caught an early run which had quietly built a split of its own; the run was discarded and repeated.

- **One metric.** Macro-F1 on the validation set, computed the same way for every model in Table 1.
- **One approach to the imbalance.** Every model uses balanced class weights derived from the training labels, ranging from 0.26 for the largest class to 9.44 for the smallest, so that a mistake on a rare disease costs proportionally more during training. The convolutional models additionally use light augmentation — horizontal and vertical flips, small rotations and small zooms — applied as preprocessing layers inside the network so that it is active during training only and cannot leak into the validation or test data. The classical models work from pre-computed features and are not augmented.
- **A sealed test set.** The 15% test partition was opened once, at the very end, for one model. Everything else in this report — every comparison, every decision — was made on the validation set. Over-fitting during development was judged from the gap between training and validation curves, not by spending the test set early.

One further point belongs here because it governs how the table in the next section should be read. Training runs in this project were carried out on cloud GPUs, and repeating an identical run in a different session produces a slightly different result. That variation has been measured at roughly ± 0.02 macro-F1. Differences between models smaller than that should not be treated as real, and Section 8 is careful to argue the final decision on more than a narrow numerical margin.

Taken together these four decisions cost very little to make and are close to impossible to retrofit. A comparison assembled from models trained on different splits, scored on different metrics, or tuned against a test set that was consulted along the way is not a comparison at all, however carefully the numbers are tabulated afterwards.

4. The ladder at a glance

The models were not tried at random. Each was chosen to answer a specific question, and the answer determined what was built next. Table 1 lists all eleven in the order they were built, scored on the shared validation set.

Model	What it is, and the question it answers	Macro-F1	Accuracy
Dummy — most frequent	Always predicts the largest class. Where does the floor sit, and is accuracy safe to use?	0.005	0.101
Dummy — stratified	Predicts by class frequency at random. A second no-skill reference point.	0.026	0.046
Logistic regression, 6 global features	Linear model on the whole-image summaries from Deliverable 1. Are those features linearly separable?	0.274	0.320
Random Forest, 6 global features	The same six features under a non-linear ensemble. What is non-linearity alone worth?	0.382	0.507
Random Forest, ~160 features	Bagged trees on colour histograms and texture. How far can hand-built features go?	0.906	0.926
Scratch CNN	Four-block network learning from pixels at 128 px. Does learning features beat designing them?	0.973	0.977
Tuned CNN	The same architecture after a 15-trial Bayesian search. Was there headroom left in the settings?	0.970	0.973
Transfer — MobileNetV2	Lightweight ImageNet backbone, fine-tuned. Do pretrained features help on this data?	0.919	0.932
Transfer — ResNet50	Larger ImageNet backbone, fine-tuned on the canonical split.	0.978	0.981
Transfer — EfficientNetB0	Compound-scaled ImageNet backbone. Does a more efficient family do better?	0.982	0.984
DenseNet-121	ImageNet backbone with dense connectivity, fine-tuned in two phases. The final model.	0.9918	0.9931

Table 1. All eleven models on the shared validation set. Macro-F1 is the decision metric; accuracy is shown for context only. No test-set figure appears here — the sealed evaluation of the final model is reported separately in Section 9.

Read from the top, the table tells a story in three movements. The first four rows establish how much of the problem can be solved without looking at where anything is in the image: not much. The fifth row shows how far careful feature

engineering can push that, and it is further than one might expect. From the sixth row onwards the models learn their own features from raw pixels, and the numbers change character — the question stops being whether the problem can be solved and becomes which of several good solutions to prefer. The next three sections take those movements in turn.

5. What the classical models taught us

The floor

The two dummy models exist to establish what a score means before any real model is judged against it. Neither learns anything. One always predicts the most common class; the other draws predictions at random in proportion to how often each class appears. Their macro-F1 scores, 0.005 and 0.026, are the numbers a model must clear to be said to have learned anything at all, and the gap between their accuracy and their macro-F1 is the clearest possible warning against reading accuracy on this dataset.

Features matter about five times more than the algorithm

The next three rows were designed to separate two things that are easy to confuse. Comparing logistic regression with a Random Forest on the same six features isolates the value of the algorithm: both see identical inputs, and the only difference is that one can model non-linear relationships and the other cannot. That change is worth roughly 0.11 macro-F1. Keeping the Random Forest but expanding the input from six features to about 160 is worth roughly 0.52.

The features are therefore worth about five times what the choice of algorithm is worth, and this is probably the most transferable thing the classical tier taught the project. It also changes how the baseline's 0.906 should be read. That is a strong result for a classical model, but the credit belongs to the inputs rather than to the ensemble: the colour histograms alone account for around 83% of the model's feature importance, against roughly 7% for the six whole-image summaries.

The richer feature set is worth describing, since it is what the deep models had to beat. It consists mainly of colour histograms computed in two colour spaces — the familiar red, green and blue channels, and the hue, saturation and value representation that separates colour from brightness — together with a set of texture descriptors derived from a grey-level co-occurrence matrix, which measures how often particular pairs of intensities appear next to one another. The six whole-image summaries from Deliverable 1 are retained alongside them. Nothing here is exotic; it is the standard toolkit a practitioner would reach for before turning to deep learning, and 0.906 is a fair account of how far that toolkit goes on this problem.

A Random Forest was chosen for these rows because it is an ensemble of the bagging family — many decision trees, each grown on a resampled subset of the data and a random subset of the features, with their votes averaged. That construction suits the job: it copes with heterogeneous features without rescaling them, it accepts class weights directly, and it reports which features it relied on, which is what made the analysis above possible. A boosting model, which builds trees sequentially so that each corrects the last, was not built. It would very likely have improved on 0.906 by a few points, but not by the 0.09 that separates the classical tier from the convolutional one, for the reason set out next.

Why the classical tier stops where it does

A colour histogram counts how many pixels of each colour an image contains. That is enough to notice that a leaf has off-colour pixels, which is often enough to know that something is wrong with it. What a histogram cannot do is say where those pixels are — whether they form a ring, a streak along a vein, or a scattering of small spots. Two diseases that discolour the same amount of tissue in different patterns produce nearly the same histogram.

The ceiling at 0.906 is therefore a property of the representation rather than of the model sitting on top of it. Histograms are spatially blind, and disease identity is spatial. No amount of additional trees, or a switch from bagging to boosting, can supply information that the inputs do not contain. Closing that gap requires a model that constructs its own inputs, and that is the next rung.

One caution belongs in the record. Among the six whole-image summaries, *file size in kilobytes* ranked as one of the most influential, at 0.20 importance in the six-feature Random Forest. File size is an artefact of JPEG compression, not a fact about plant biology. It is the earliest sign in this project that a model can score well by reading properties of the photograph rather than properties of the plant — a concern that returns, measured directly, in Section 11.

6. What the convolutional network taught us

A convolutional network does not receive a summary of the image. It receives the pixels, and it learns for itself which small local patterns are worth detecting — an edge, a boundary between healthy and dead tissue, the concentric banding of a particular lesion — and then which arrangements of those patterns indicate which disease. The spatial information that a histogram discards is exactly what this architecture is built to keep.

The effect is immediate. The scratch CNN, four convolutional blocks trained from random initialisation at 128 pixels, reaches 0.973 macro-F1 with 263,654 parameters. That is a jump of 0.067 over the strongest classical model, and it is achieved with fewer stored numbers than the classical pipeline extracts features per image. The prediction made in Deliverable 1 — that the disease signal is local and spatial, and that a model would have to see structure rather than summaries — is paid out in full by this single row of the table.

Two properties of the architecture do the work. The first is locality: each filter looks at a small patch at a time, which matches the fact that a lesion is a local event on an otherwise ordinary leaf. The second is that the same filter is applied everywhere in the image, so a pattern learned from a lesion near the tip is recognised when it appears near the base. Together these let the network learn a small vocabulary of lesion appearances and then detect them anywhere, which is exactly the invariance the problem calls for and exactly what a global histogram throws away.

Was there anything left in the settings?

Having got a strong result from an architecture chosen by hand, the obvious question is whether the configuration was lucky or good, and whether a systematic search would find something better. That is the entire purpose of the tuned CNN: not to squeeze out an extra fraction of a point, but to establish whether the baseline configuration had headroom that was being left on the table.

A Bayesian search using keras-tuner ran fifteen trials over five hyper-parameters — learning rate, dropout, the width of the dense head, the width of the convolutional base, and the strength of augmentation. Bayesian optimisation was used rather than an exhaustive grid because it builds a model of which regions of the search space look promising and concentrates its trials there, which matters when each trial costs GPU time. To keep the search affordable, trials were ranked on a stratified subsample of the training data and the winning configuration was then retrained in full.

The answer was that there was no headroom. The search rediscovered the hand-chosen configuration on four of the five parameters, and the retrained winner scored 0.970 against the original 0.973 — a difference well inside run-to-run variation. This is a useful negative result rather than a wasted experiment. It says that the remaining errors at this level are not caused by mis-set dials, and therefore that further progress had to come from a different architecture rather than from better tuning of this one. That conclusion is what sent the project to transfer learning.

7. What transfer learning taught us

Transfer learning starts a model from weights already trained on a different, much larger dataset — here ImageNet, roughly 1.4 million photographs of everyday objects and scenes. The idea is that the early layers of any vision model learn generic things: edges, corners, textures, colour gradients. Those are as useful for a leaf lesion as for a lamppost, so rather than learning them again from scratch, the model inherits them and only has to learn what is specific to the task.

Four backbones were fine-tuned on the canonical split: MobileNetV2, ResNet50, EfficientNetB0 and DenseNet-121. All four were trained under a single standardised protocol, which was itself the most consequential piece of tuning in the project.

The four were not chosen arbitrarily. MobileNetV2 is the lightweight option, designed for phones, and was the cheapest way to test whether pretrained features helped at all. ResNet50 is the long-standing reference architecture against which most others are reported. EfficientNetB0 represents a family designed by scaling depth, width and resolution together rather than one at a time. DenseNet-121 reuses features aggressively, with every layer inside a block receiving the outputs of all those before it, which lets it be deep without being wide. Between them they cover most of the design space a practitioner would consider, at four different points on the size-versus-capacity trade-off.

- **Two phases.** The backbone is frozen first and only the new classification head is trained, at a relatively high learning rate. Once the head is sensible, the backbone is unfrozen and the whole network is trained together at a learning rate two orders of magnitude lower, so that the inherited features are refined rather than overwritten.
- **Normalisation layers stay frozen.** The BatchNorm layers keep their inherited running statistics throughout. Letting them adapt on small effective batches destabilises fine-tuning, and freezing them was the single change that made these runs reliable.

- **Verification that fine-tuning happened.** Each run asserts that the number of trainable parameters actually rises when the backbone is unfrozen. Without that check, a configuration error can leave the second phase training nothing at all while still producing a plausible-looking score.

Under that protocol, three of the four backbones cleared the scratch CNN and DenseNet-121 cleared it clearly, reaching 0.9918. The tier as a whole delivered a smaller improvement than the literature on transfer learning might suggest, and the reason is instructive: transfer learning helps most when training data is scarce, and 38,013 training images of a visually uniform kind is not scarce. A network trained from scratch has enough data here to learn good features on its own, so the inherited head start is worth less than it would be otherwise.

That observation looks like a limitation of transfer learning on this dataset. Section 8 argues that it is in fact the strongest reason to prefer the pretrained model, because the dataset this tool will eventually have to handle is not this one.

8. Choosing the final model

The decision rests on validation performance. Across all eleven models, DenseNet-121 has the highest macro-F1 on the shared validation set, at 0.9918 against 0.973 for the scratch CNN, its nearest architecturally distinct rival. That is the primary and simplest reason it was taken forward.

The margin is 0.019, which is close to the ± 0.02 run-to-run variation documented in Section 3, so it would be careless to rest the decision on the headline number alone. Four further considerations support it, and the first is the strongest.

It is much the better base for a field version

This is the main argument for the final model, and it follows directly from the observation that closed Section 7.

Pretrained weights pay off most when training data is scarce. PlantVillage is not scarce, which is why transfer learning delivered a smaller advantage here than it usually does. But the version of this tool that matters is the one that works on leaves photographed on living plants, in daylight, against soil and other foliage — and that data *will* be scarce. Field imagery arrives in the hundreds or low thousands of images, not the tens of thousands.

In that regime the two candidates are not comparable. DenseNet-121 carries 7.1 million parameters of general visual structure learned from 1.4 million varied natural photographs, and adapting it to a small field dataset is a shallow re-fine-tune of a network that already knows what daylight, occlusion and cluttered backgrounds look like. The scratch CNN carries 263,654 parameters learned exclusively from detached leaves on a plain grey backdrop; adapting it to field conditions is much closer to starting again, on a dataset far too small to start from.

Choosing DenseNet-121 therefore costs nothing in laboratory performance and keeps open the one route that leads to a deployable tool. Section 13 makes that route the project's concrete next step.

It is worth stating the counterfactual, because it clarifies what the argument does and does not claim. If the objective were only to post the best number on this benchmark and stop, the case would be much weaker: the margin over the scratch CNN is close to noise, the smaller model is twenty-seven times lighter, and simplicity would be a reasonable tie-breaker. The objective is not that. It is a tool that eventually works on a photograph taken in a field, and under that objective the two candidates are not close.

The errors it removes are the ones that were hardest to remove

Against the scratch CNN, the final model reduces validation errors by a factor of 3.4, from 190 to 56. What matters more than the count is where those errors were. The final model holds a minimum per-class recall of 0.9481, with only one class of 38 below 0.95 and 20 of 38 recalled perfectly. Because a single class now sits below that threshold, every class that the scratch CNN's error analysis had named as hard — Tomato Early blight, Target Spot, Corn Northern Leaf Blight, Apple scab — has been lifted clear of it.

A model that has merely got lucky improves where there was already room to improve. Repairing the tail is what macro-F1 was chosen to reward in the first place, and it is what a diagnostic tool needs, because the disease a grower most needs identified is usually not the common one.

The result holds on data no model had read

The sealed evaluation in the next section returns a test macro-F1 of 0.9919 against a validation figure of 0.9918, a difference of +0.0001, with a 95% confidence interval of [0.9896, 0.9941]. The margin is therefore not an artefact of the checkpoint having been selected against the validation set.

What is left over is largely not fixable by choosing a different model

Section 10 shows the 57 remaining test errors resolving into a small number of recurring pairs, most of which are either genuine biological ambiguity or low-cost confusions between two hosts of the same pathogen. The error budget on this dataset is close to its irreducible part, which is a further reason to stop optimising here and change the data instead.

What did not decide it

It is worth saying what was considered and set aside. The interpretability work described in Section 11 measures where each model looks, and it does not separate the two candidates in a way that would justify overriding their scores: both attend preferentially to the leaf, and the small difference between them runs in the pretrained model's disfavour rather than its favour. That measurement is reported in full because it is informative about the models, but it plays no part in the selection, and the report does not use it to argue for the model that was chosen.

What the choice costs

The final model has 7,076,454 parameters against the scratch CNN's 263,654, and takes 72 minutes to train on a T4 GPU against a few minutes. The argument from simplicity, which is a real argument and was taken seriously, loses here on merit. Inference cost is not a differentiator — both models are fast enough for interactive use and small enough to serve inside a Streamlit application — and the training cost is paid once.

9. The final model, and the sealed test set

What the final model is

DenseNet-121 is a convolutional network whose distinguishing feature is dense connectivity: within each block, every layer receives the outputs of all preceding layers. That reuse lets the network be deep without being wide, which is why it reaches this depth at a little over seven million parameters.

It was initialised with ImageNet weights and trained on the canonical split at 128 pixels with a batch size of 32 and balanced class weights, following the two-phase protocol described in Section 7. The first phase trained the classification head alone, with the backbone frozen, at a learning rate of 1e-3 for five epochs. The second unfroze 306 of the network's 427 layers — all 121 BatchNorm layers were kept frozen — and continued at 1e-5 with early stopping, which triggered at epoch 19 with the best weights taken from epoch 15. Trainable parameters rose from 38,950 to 6,909,158 between phases, confirming that fine-tuning engaged, and balanced class weights were verified as passed in both phases. Total training time was 72 minutes on a T4.

The sealed evaluation

The test partition had not been read by any model before this point. It was opened once, for this model, and has not been opened since. The evaluated checkpoint is identified by MD5 hash afe3478450c41d91a6ac1e57a2ada44d, so the result is tied to a specific file rather than to a description of one.

DenseNet-121	Validation (8,146)	Test (8,146, sealed)
Macro-F1	0.9918	0.9919 95% CI [0.9896, 0.9941]
Accuracy	0.9931	0.9930
Errors	56 of 8,146	57 of 8,146 (0.70%)
Minimum per-class recall	0.9481	0.9481

Table 2. The final model on the validation set and on the sealed test set. The confidence interval is a bootstrap 95% interval over the test predictions.

The confidence interval is worth a word, because a single number computed once invites more confidence than it deserves. It was obtained by bootstrapping: the test predictions are resampled with replacement many times, macro-F1 is recomputed for each resample, and the interval covers the middle 95% of those values. It describes how much the score would be expected to move if a different sample of 8,146 leaves of the same kind had been drawn. At ± 0.0023 it is narrow, which is a consequence of the test set being large.

The gap between validation and test is +0.0001 — the model scores fractionally better on data it has never been evaluated against than on the data used to choose its checkpoint. There is no measurable over-fitting. This is precisely the result that the sealed protocol exists to make believable: had the test set been consulted during development, a

small gap like this would carry very little information, because the test set would by then have become part of the development loop.

Set against the ladder, the final model reduces the error rate of the strongest classical model by roughly a factor of eleven and that of the scratch CNN by 3.4. In absolute terms it misidentifies 57 leaves out of 8,146 — about one in 143.

10. Where the remaining errors are

Fifty-seven errors is a small enough number to examine individually, and doing so is more informative than the headline score. They are not scattered evenly across the problem. They fall into 31 distinct pairs of true and predicted class, and the ten largest pairs account for 63% of all errors. The five largest are below.

True class → predicted class	Images	Share of class	Share of all errors
Tomato Spider mites → Tomato Target Spot	9	3.6%	15.8%
Tomato Yellow Leaf Curl Virus → Tomato Bacterial spot	8	1.0%	14.0%
Tomato Late blight → Potato Late blight	5	1.8%	8.8%
Tomato Late blight → Tomato Early blight	4	1.4%	7.0%
Corn Cercospora Gray leaf spot → Corn Northern Leaf Blight	4	5.2%	7.0%

Table 3. The five largest confusion pairs on the sealed test set. Share of class is the pair's size relative to that true class's test images; share of all errors is relative to all 57.

Three patterns run through them, and each means something different for how the tool would behave in use.

- **Confusion between similar diseases of the same crop.** Spider mite damage read as Target Spot; Late blight read as Early blight. These are the fine-grained core of the task — distinctions that a non-specialist would also struggle to make from a photograph — and they are the errors one would most expect to survive at this level of accuracy. They are also the errors with real diagnostic cost, since the treatment differs.
- **Confusion between crops sharing one pathogen.** Tomato Late blight predicted as Potato Late blight, five images, with one error in the reverse direction. Tomato and potato are both Solanaceae, and *Phytophthora infestans* produces genuinely similar lesions on both. Judged as a diagnosis rather than as a label, this is the least costly error in the table: the pathogen is identified correctly and only the host is wrong, and the host is something the person holding the phone already knows.
- **One stable ceiling.** Corn Cercospora leaf spot mistaken for Corn Northern Leaf Blight — four images, 5.2% of that class. This is the class that sets the minimum per-class recall at 0.9481, and that recall is 0.9481 on validation *and* on test, which makes it a stable property of the data rather than sampling noise. The same confusion appeared in every model built in this project, from the Random Forest upward. At 128 pixels the two lesion types are close to indistinguishable, and no amount of further training will separate them at that input resolution.

Error analysis was more productive earlier in the project than it is at this level. When the scratch CNN was the strongest model, per-class recall named a hard set of four classes, and those turned out to be exactly where the pretrained backbones gained their advantage. At 0.9919 the remaining signal is thinner, and what the analysis now supplies is two specific, testable proposals rather than a general direction: the Corn pair is a resolution hypothesis, testable by retraining that crop's classes at 256 pixels, and the Tomato Spider mites pair — the single largest at 15.8% of all errors — is a candidate for augmentation targeted at that class rather than more epochs applied to all of them.

It is also worth noting what does not appear in the table. None of the five largest pairs involves a diseased leaf being classified as healthy, which is the failure mode with the highest cost in practice — a grower told that a sick plant is fine will act on that, or rather will not act at all. That such errors are absent from the top of the list is a meaningful property of the model, and it is a direct consequence of choosing a metric that would not let the rare and difficult classes be averaged away during development.

11. Looking inside the model

A model that is right 99.3% of the time is still a model that gives an answer without a reason. For a diagnostic tool that is not quite good enough: a grower shown a disease name has no way to tell a confident, well-founded prediction from a lucky one. This section describes what can be seen inside the final model when it makes a decision, and what that reveals about both its strengths and its limits.

What Grad-CAM shows

Grad-CAM recovers something close to a reason. It reads the last convolutional layer of the network — the deepest point at which the information still has a location in the image — and measures how strongly each region there pushed the prediction towards the class the model actually chose. The output is a heat map laid over the original photograph: warm where the evidence for the decision came from, cool where it did not.

It answers one question well — where did the model look? — and should not be stretched further. It does not say what would have changed the model's mind, and it does not show that the model understands anything about plant pathology. It is a diagnostic instrument, not a proof.

Turning heat maps into a number

Heat maps are persuasive and easy to over-read, so the question was converted into a measurement. PlantVillage ships a segmented version of every image — the same photograph with the background masked out, produced by the dataset's own authors — which supplies a free outline of the leaf. For each image, the share of the heat map falling inside that outline was divided by the share of the frame the leaf occupies. The resulting figure, called concentration here, reads simply: 1.0 means no preference at all, above 1.0 means the model concentrates on the leaf, and below 1.0 means it preferentially reads a background that contains no disease information whatsoever.

The measurement covers the 7,967 validation images that have a segmented counterpart, 97.8% of the validation set. Leaf area averages 47.4% of the frame, identically for both models, since that is a property of the photographs rather than of the networks.

Two things make this a fair probe. The masks were produced by the dataset's authors rather than by us, so they are not tuned to flatter either model, and neither network ever saw them — all training used the unsegmented colour images. The measurement is therefore genuinely external to the models being measured, which is what makes the comparison between them meaningful rather than circular.

Model	Attention on leaf	Concentration (native)	Concentration (matched 4×4)
Scratch CNN	56.7%	1.226	1.169
DenseNet-121	53.3%	1.148	1.134
Difference	-3.4 pp	-0.078	-0.034

Table 4. On-leaf attention across 7,967 masked validation images. Concentration is attention share divided by leaf area. The matched column recomputes both models' heat maps on a common 4 × 4 grid; the 95% intervals on the difference are [-0.090, -0.066] native and [-0.043, -0.025] matched.

What it found

Both models prefer the leaf. The scratch CNN places 56.7% of its attention on a leaf occupying 47.4% of the frame, and the final model places 53.3% there. Both sit comfortably above 1.0 concentration, so neither is reading the backdrop in preference to the plant. Both also leave a substantial share of their attention on a plain grey background that carries no disease information at all, which is a useful corrective to scores in the high nineties.

The final model's concentration is slightly the lower of the two, and the difference is statistically real. Part of it is an artefact of measurement: DenseNet-121's last convolutional layer produces a 4 × 4 grid at this input size, where the scratch CNN's produces 16 × 16, and a coarser map cannot help but spill across a leaf edge. Recomputing both on a common 4 × 4 grid controls for that, and the difference more than halves but does not disappear. What this means practically is that the final model gives no evidence of being less dependent on the laboratory backdrop than the from-scratch network, and no claim of that kind is made for it anywhere in this report. Whether it would hold up in a field is an open question that only field images can answer.

The useful finding: attention tracks correctness

Splitting concentration by outcome produces the most practically valuable result of the interpretability work. On the images the final model classifies correctly, concentration averages 1.135; on the images it gets wrong, 1.009. The difference is +0.126, with a 95% interval of [+0.053, +0.203] and a Mann-Whitney p-value of 7.1×10^{-4} . The scratch CNN shows the same effect more weakly: 1.171 against 1.080, a difference of +0.091.

A concentration of 1.009 is no leaf preference at all. On the images the final model gets wrong, its attention is spread as though the leaf were not there. Misclassification here is not a subtle judgement between two plausible diseases — it coincides, measurably, with the model reading the wrong part of the frame.

That is directly usable. It means attention concentration can be computed at inference time and used as a confidence signal, flagging predictions whose heat map sits off the leaf for human review, without needing a ground-truth label to check them against. The effect is stronger in the final model than in the scratch CNN, which adds a fifth practical reason to the four given in Section 8, and it is a concrete route to a safer diagnostic tool rather than merely a higher score.

12. Limitations

The following are the things this report does not establish, stated so that a reader does not have to infer them.

- **Every number here is a laboratory number.** PlantVillage is detached leaves photographed flat against a plain background under even light. A deployed system sees leaves on living plants, in variable daylight, against soil and other foliage, often partly occluded and rarely centred. The published literature on this dataset reports large performance drops under those conditions, and nothing in this report contradicts that expectation.
- **No field-robustness claim is made for the final model.** The attention measurement in Section 11 gives no evidence that the pretrained model depends less on the laboratory backdrop than the from-scratch one. The argument in Section 8 is about which model is the better *starting point* for a field version, not about which performs better in a field today. That distinction matters and is not being blurred.
- **The attention comparison is partly confounded by resolution.** Matching both models on a 4×4 grid controls for it and the effect survives, but it more than halves. A cleaner test would compare heat maps drawn from an intermediate DenseNet block at 8×8 or 16×16 .
- **No boosting model was built.** Section 5 gives the reasoning — the classical ceiling is set by the features rather than by the ensemble — but the comparison between bagging and boosting on this feature matrix remains a gap.
- **Run-to-run variation is roughly ± 0.02 macro-F1.** Differences smaller than that between any two rows of Table 1 should not be read as meaningful. This is why the argument in Section 8 rests on error structure and on the field-adaptation case rather than on the headline margin.
- **The transfer models were not compared at their intended input sizes.** All four backbones were fine-tuned at 128 pixels, which is a fair comparison at fixed compute but not a fair comparison of the architectures as their authors designed them.
- **Not every intermediate artefact survives.** Some models were trained across several cloud sessions, and a small number of early checkpoints and notebooks were lost when runtimes were recycled. Where a figure in Table 1 comes from a surviving artefact and an earlier document records a different one, the surviving artefact is the figure of record. This affects no ordering in the table and no conclusion in Section 8, but it is the reason the final model's checkpoint is identified by hash.

13. Outlook and conclusion

What the project learned

The most transferable finding is not the headline score but the decomposition beneath it. On this task, the representation dominates the choice of model: the same Random Forest gains 0.52 macro-F1 from richer features and only 0.11 from non-linearity, and the entire classical tier is capped by the spatial blindness of colour histograms rather than by the capacity of the ensemble built on them. Convolution's advantage here is specific and nameable — it is the ability to say where a discoloured pixel sits, not merely that one exists.

The second finding is that accuracy and attention are separable properties. The final model scores highest and attends to the leaf marginally less than a model scoring lower, so a project that selects on the score should not assume it has also bought better localisation. The useful corollary is that attention still tracks correctness within each model, and does so most sharply in the final one, which turns Grad-CAM from a presentation device into a runtime confidence signal.

The third is that a saturated benchmark is a warning rather than a success. At 0.9919 on sealed test data, with twenty of thirty-eight classes recalled perfectly, further gains on this dataset are close to meaningless. The remaining errors are either genuine biological ambiguity, as in the Corn pair, or confusions whose practical cost is negligible, as in the Tomato-to-Potato Late blight pair. The task in front of this model is not a higher number.

What we would do next

This is a capstone project with a fixed timetable and no budget for data collection, so none of what follows is work this team is in a position to carry out. It is set down because the limitations above point at specific, answerable questions

rather than vague ones, and because anyone taking this model further should not have to rediscover them. If the resources existed and this tool had to work in a field rather than on a laboratory bench, the following is the order in which we would tackle it.

- Acquire or borrow field imagery — leaves in situ, phone cameras, mixed backgrounds and lighting — and measure the drop rather than argue about it. This is by a wide margin the most informative thing left to do, and it is the step for which the final model was chosen.
- Adapt the final model to that imagery with a shallow re-fine-tune. Unfreezing 306 of 427 layers maximises fit to the laboratory distribution; a shallower unfreeze that preserves more of the inherited general structure is the natural configuration to test against field data once it exists.
- Build the gradient-boosting model on the existing feature matrix and close the bagging-versus-boosting comparison. The features already exist, so this is a fit call rather than a pipeline.
- Retrain the Corn classes at 256 pixels to test the resolution hypothesis behind the only class below 0.95 recall, and apply targeted augmentation to the Tomato Spider mites class rather than more epochs to all of them.
- Implement attention concentration as a runtime confidence signal, flagging predictions whose heat map sits off the leaf for human review.
- Consider a pretraining corpus closer to the domain. What ImageNet contributes here is low- and mid-level texture and edge structure rather than its semantic categories, which are of little use for flat leaf scans. Weights pretrained on in-situ plant photography would be the principled fix for the laboratory-to-field gap.

Conclusion

The modelling objective set in Deliverable 1 has been met. The final model identifies all 38 crop-and-disease classes from a single leaf photograph at 0.9919 macro-F1 on data it had never seen, with a validation-to-test gap of +0.0001 and a worst class still recalled at 0.948. The comparison behind that choice was made on one split, with one metric, under one set of conditions, and the reasoning is recorded here rather than assumed.

What remains is not a better model on this dataset. It is a dataset that resembles the problem, and a model chosen so that it can be adapted to one.

The honest summary of the position is that the modelling question has been answered and the deployment question has not been asked yet. Everything in Sections 4 to 11 concerns how well a model can distinguish 38 conditions in photographs taken under controlled conditions, and the answer is: very well, with the residual difficulty concentrated in a handful of genuinely ambiguous pairs. Whether the same model can distinguish them on a leaf still attached to a plant, in ordinary light, is a separate question that this dataset cannot answer. The value of the work done here is that it leaves that question in a good position to be asked.

Deliverable 2 — Modelling Report · Plant Recognition · PlantVillage

Technical Appendix

This appendix records, for each of the eleven models in Table 1, what was actually built and run: the architecture, the hyper-parameters, the compute it consumed, the artefacts it produced, and the technical reasoning behind the choices. It is written to be consulted rather than read straight through. Part I argues; this half is the evidence behind the argument, at the level of detail a reader would need in order to rebuild any of these models and get the same numbers back.

Space is allocated by how much of the project's conclusion rests on each model. The two dummy models and the three transfer backbones that were not selected are short records — enough to reproduce them and no more. The Random Forest on roughly 160 features and the from-scratch convolutional network are treated at length, because the comparison between them carries the central finding about representation. DenseNet-121, the final model, has the longest section, because it is the model whose behaviour the project is claiming to understand.

Every figure here is drawn from the run artefacts, and every number is traceable to a notebook output or a saved CSV. Where a figure could not be reconstructed from what survives, the section says so rather than estimating. Most figures were redrawn in a plain style; six are reproduced **unedited from the notebooks**, in the notebooks' own colour schemes, and are labelled as such in their captions.

A. Common ground — what every model shared

Eleven models produce eleven numbers, and those numbers only form a comparison if the conditions underneath them are identical. This section records those conditions once so that the model records in Section B do not have to repeat them. Anything a model did *differently* is stated in its own record; anything not stated there follows what is written here.

A.1 Compute environment

All work ran in Google Colab. The deep models used the standard GPU runtime — an NVIDIA Tesla T4 with 16 GB of GDDR6 — and the classical models ran on the CPU runtime, since scikit-learn does not use the GPU. TensorFlow 2.20.0 on Keras 3 was the deep-learning stack throughout; scikit-learn supplied the dummy classifiers, the logistic regression and both Random Forests, and scikit-image supplied the colour-space conversion and the grey-level co-occurrence matrix used for texture features.

Two consequences of that environment shaped the project more than they should have, and both are worth stating plainly. The first is that a Colab runtime is recycled without warning, and the file system goes with it. Several trained checkpoints and one set of split CSVs were lost that way. The response was a `USE_DRIVE` toggle in every notebook and a convention of saving the model to Drive *immediately after training and before evaluation*, so that a wipe during the evaluation cell could not cost an hour of GPU time. The second is that identical runs on cloud GPUs do not reproduce exactly. Non-deterministic kernel scheduling, combined with a shuffled input pipeline, moves the final macro-F1 by roughly ± 0.02 between sessions. That figure is the reason Part I refuses to treat small differences between rows of Table 1 as real.

The Kaggle dataset — 2.04 GB, 54,305 images — was downloaded fresh into each runtime using credentials read from Colab Secrets, with a retry loop, because the secrets service intermittently fails on the first call. Download took between eight and twenty seconds depending on session throughput.

A.2 The canonical split

The split was constructed once and then distributed as three CSV files rather than regenerated locally by each notebook. All 54,305 image paths were enumerated from the untouched `color` rendering of the Kaggle mirror and divided by two successive stratified `train_test_split` calls with seed 42.

Train	38,013 images (70.0%)
Validation	8,146 images (15.0%)
Test	8,146 images (15.0%) — sealed until the final evaluation
Stratified by	the 38-class label
Seed	42
Fingerprint	SPLIT_ID 9e33ec57c1ec (MD5 over the three CSVs)
Distributed as	step3_baseline/split_train.csv, split_val.csv, split_test.csv

Table A.1. The canonical split. Every notebook asserts that all 38 classes are present in each partition and that the three sets are pairwise disjoint before it proceeds.

From Deliverable 2 onwards each notebook recomputes the fingerprint from the CSV bytes at startup, prints it, and halts if it does not read `9e33ec57c1ec`. That guard exists because it was needed: an early transfer-learning run silently rebuilt a split of its own when it could not find the CSVs, and a ResNet50 run was later found to have trained on a seed-123 split of its own construction, which had put roughly 5,700 sealed test images into its training data. Both runs were discarded. The fingerprint gate is cheap and it is the only thing standing between a shared benchmark and eleven incomparable numbers.

A.3 The image pipeline

Every convolutional model reads images through the same `tf.data` pipeline. Files are read from disk, decoded with `tf.io.decode_image` at three channels, and resized bilinearly to 128×128 . Pixels are left in the 0–255 range at this point; normalisation is applied *inside* the model as a layer, not baked into the data. That choice comes from Deliverable 1 and it matters for two reasons: a saved `.keras` file then carries its own preprocessing and cannot be fed wrongly scaled input by accident, and different backbones can apply their own `preprocess_input` convention without the pipeline having to know which model it is feeding.

The training dataset is shuffled with a buffer of up to 2,048 and reshuffled each epoch; the validation and test datasets are **not** shuffled, so that the prediction array returned by `model.predict` lines up positionally with the label array used to score it. Batching is 32 throughout, and `prefetch(AUTOTUNE)` overlaps loading with computation.

The classical models do not use this pipeline. They read images with PIL at full resolution for the six global summaries, and at a downsampled 64×64 for the histogram and texture features, and they write the resulting feature matrix to a compressed `.npz` cache so that a re-run costs seconds rather than minutes.

A.4 Class weights

The dataset is imbalanced by roughly 36 to 1 — Orange Huanglongbing has 5,507 images, healthy Potato has 152. Every model in the comparison corrects for this the same way, with balanced class weights computed from the **training** split only:

$$w(c) = n_{\text{total}} / (n_{\text{classes}} \times n_c)$$

which yields weights from 0.26 for the largest class to 9.44 for the smallest. The three most heavily weighted classes are healthy Potato at 9.4, Apple Cedar apple rust at 5.2 and healthy Peach at 4.0. For the Keras models the weight dictionary is passed to `model.fit` as `class_weight`; for the scikit-learn models it is `class_weight="balanced"`, which recomputes the same quantity internally on the data it is fitted to.

Because the weight range is a deterministic function of the training split, it doubles as a cheap cross-notebook check: two runs printing the same 0.26–9.44 range and the same three top-weighted classes are demonstrably reading the same split, even when one of them belongs to a teammate and its fingerprint was never printed.

A.5 Augmentation

Augmentation applies to the convolutional models only. It is defined as a small `Sequential` of Keras preprocessing layers placed inside the network, immediately after the input:

RandomFlip	"horizontal", seed 42
RandomRotation	0.05 ($\approx \pm 18^\circ$)
RandomZoom	0.05

Table A.2. The augmentation block, identical in the scratch CNN, the tuned CNN and all four transfer models. The tuned CNN exposes the rotation and zoom magnitude as a searchable hyper-parameter; every other model uses 0.05.

Defining augmentation as layers rather than as a dataset transformation is what makes the train-only guarantee structural rather than procedural. Keras preprocessing layers are active when `training=True` and are pass-throughs at inference, so validation and test images cannot be augmented by accident — there is no code path that would do it. The classical models operate on pre-computed feature vectors and are not augmented at all, which is the correct treatment: augmenting a colour histogram by flipping the image it came from produces the same histogram.

A.6 How the metrics were computed

Macro-F1 is `sklearn.metrics.f1_score(y_true, y_pred, average="macro", zero_division=0)` — the unweighted mean of the 38 per-class F1 scores, with a class that is never predicted contributing zero rather than raising. Accuracy is the plain fraction correct. Per-class recall is `recall_score(..., average=None)` over the 38 labels in canonical sorted order, which is the order `sorted(os.listdir(color_dir))` returns and which every notebook re-derives rather than hard-codes.

Predictions are always the `argmax` of the softmax output. No probability threshold, no top-k, no per-class calibration, and no test-time augmentation was used anywhere in this project. The confusion matrix is `confusion_matrix(y_true, y_pred, labels=range(38))` and is displayed row-normalised, so its diagonal reads directly as per-class recall.

The bootstrap confidence interval reported for the final model resamples the 8,146 test predictions with replacement, recomputes macro-F1 on each resample, and takes the 2.5th and 97.5th percentiles of that distribution. It describes sampling variation in the test set, not run-to-run variation in training — those are different quantities and the report keeps them separate.

A.7 Reproducibility conventions

Every notebook follows the same house pattern, which is worth recording because it is what made an eleven-model comparison tractable across four people and several weeks.

- **One configuration block.** Everything tunable — image size, batch size, epochs, learning rates, paths, the smoke-test flag — sits in a single cell near the top. Nothing is configured further down.
- **A `SMOKE\_TEST` flag.** Set to `True`, the notebook caps the data at a few dozen images per class and runs two epochs, giving a two-minute end-to-end dry run that exercises every cell including the save. It writes to separate filenames so a smoke run cannot overwrite a real checkpoint.
- **Skip-guards on expensive cells.** Feature extraction and training check for a cached artefact first and load it if present, so a re-run after a disconnect resumes rather than restarts.
- **Assertions, not comments.** Pixel range after loading, split disjointness, class count, and — for the transfer models — the trainable-parameter count after unfreezing are all asserted in code. A silent misconfiguration that still produces a plausible-looking score is the failure mode these guard against, and at least three such failures were caught this way.
- **Artefacts written to a per-model directory.** Each model saves its checkpoint, its results CSV, its per-class recall CSV, its run configuration as JSON and its figures to its own folder, so the comparison table can be assembled from files rather than from screenshots.

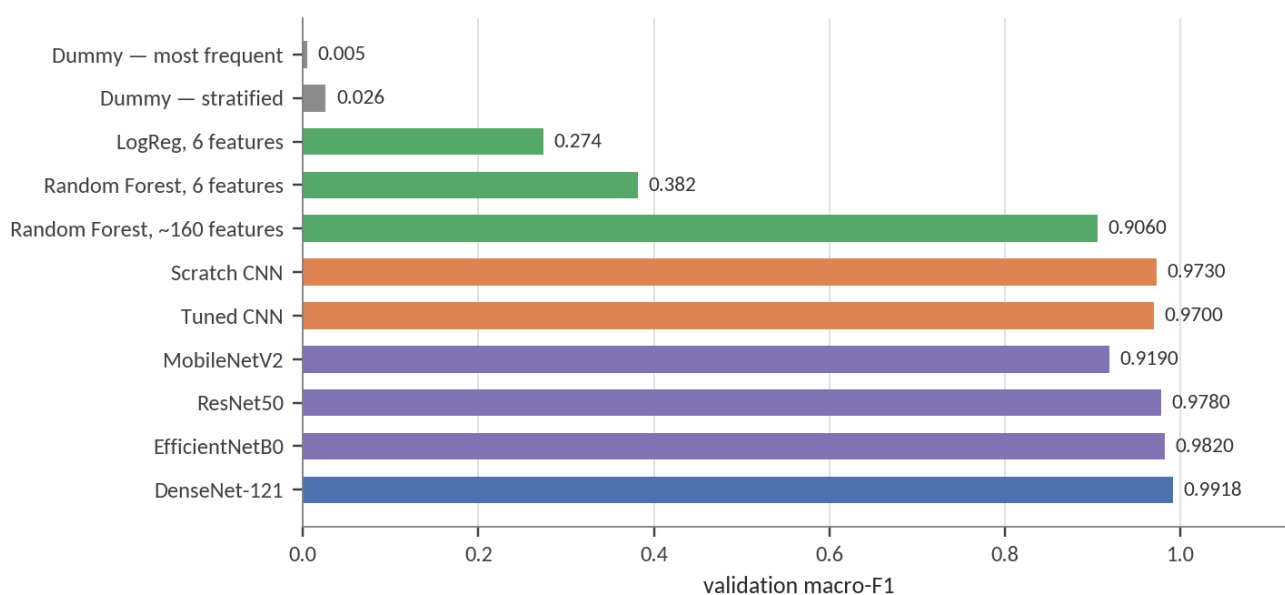


Figure A.1. All eleven models on the shared validation set, in the order they were built. The two models discussed at length in this appendix — the Random Forest on ~160 features and the scratch CNN — are shown in dark grey; the final model in black.

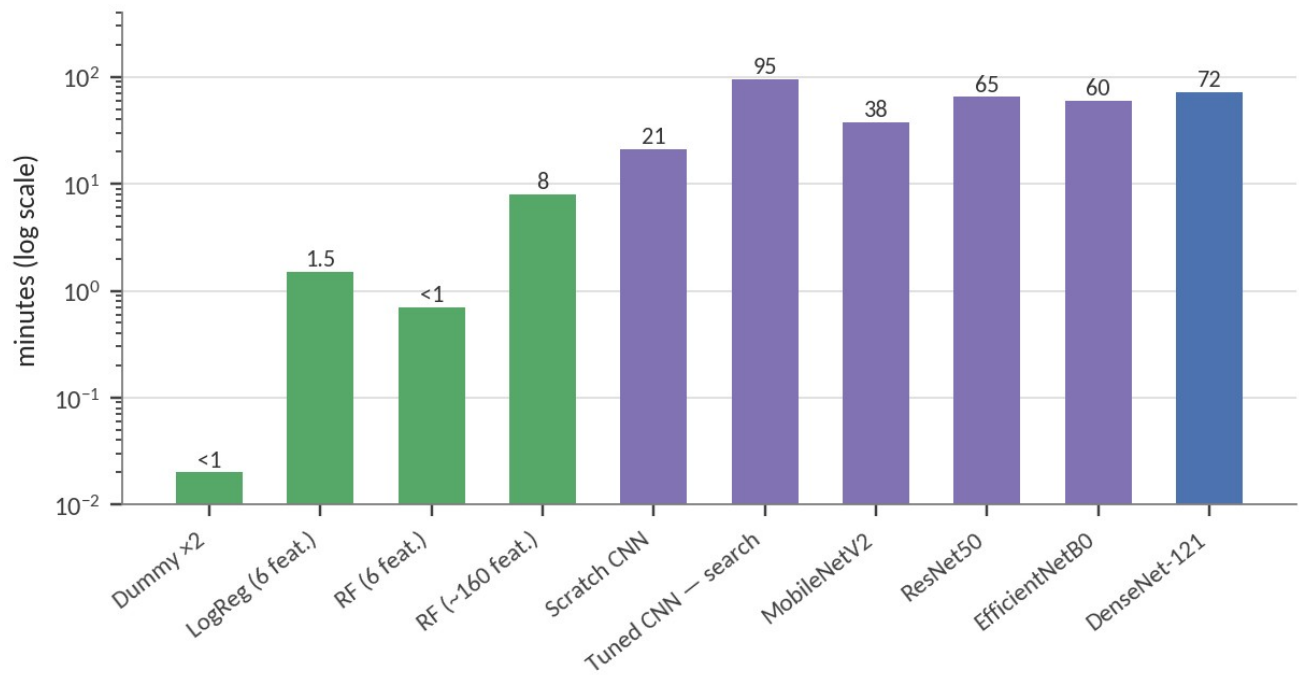


Figure A.2. Wall-clock training cost, log scale. Classical models ran on the Colab CPU runtime, deep models on a T4. The tuned CNN's bar is the fifteen-trial search, not a single fit. Figures for the classical models include feature extraction, which dominates their cost.

B. Model records

B.1 Dummy — most frequent

Implementation	<code>sklearn.dummy.DummyClassifier(strategy="most_frequent", random_state=42)</code>
Input	none — fitted against a zero column, features ignored by construction
Prediction	Orange Huanglongbing (Citrus greening) for every image
Validation macro-F1	0.005
Validation accuracy	0.101
Compute	under one second, CPU
Notebook	Step3_1_PlantVillage_Baseline_Models.ipynb

This model exists to fix the floor, and its value is entirely in the gap between its two numbers. It predicts the largest class every time. Orange Huanglongbing holds 826 of the 8,146 validation images, so accuracy is $826 / 8,146 = 0.1014$ — a figure that looks like a tenth of the problem solved by a model that has solved none of it.

Macro-F1 exposes that directly. Only one of the 38 classes gets a non-zero F1: precision 0.1014, recall 1.000, $F1 = 0.184$. The other 37 score zero. The unweighted mean is $0.184 / 38 = 0.0048$, which rounds to the 0.005 in Table 1. The arithmetic is worth spelling out because it is the whole argument for the choice of metric: on this dataset, accuracy can be an order of magnitude larger than macro-F1 for a model with no skill whatsoever.

B.2 Dummy — stratified

Implementation	<code>sklearn.dummy.DummyClassifier(strategy="stratified", random_state=42)</code>
Input	none — draws labels at random in proportion to training frequency
Validation macro-F1	0.026
Validation accuracy	0.046
Compute	under one second, CPU
Notebook	Step3_1_PlantVillage_Baseline_Models.ipynb

A second no-skill reference, and a more informative one than the first for a multi-class problem. Where the most-frequent dummy concentrates all its mass on one class, this one spreads predictions across all 38 in proportion to how often they appear, so every class receives some correct predictions by chance. Its expected accuracy is the sum of the squared class frequencies, which for this distribution is about 0.046, and its macro-F1 of 0.026 is the level a genuinely random guesser reaches on 38 imbalanced classes.

Note that its accuracy is *lower* than the most-frequent dummy's while its macro-F1 is five times higher. Two no-skill models can be ranked in opposite orders by the two metrics. That inversion, more than any argument, is why every comparison in this project is made on macro-F1.

B.3 Logistic regression on six global features

Features	file_size_kb, brightness, mean_R, mean_G, mean_B, green_frac
Preprocessing	StandardScaler fitted on the training rows only
Model	LogisticRegression(max_iter=2000, class_weight="balanced", n_jobs=-1), multinomial
Fitted on	a stratified sample of up to 300 training images per class
Scored on	the full 8,146-image validation set
Validation macro-F1	0.274
Validation accuracy	0.320
Compute	≈1.5 min CPU, dominated by feature extraction
Notebook	Step3_1_PlantVillage_Baseline_Models.ipynb

The six features are the whole-image summaries carried forward unchanged from Deliverable 1. Five are trivial reductions of the pixel array — mean brightness and the three channel means, plus the green fraction $g / (r + g + b)$ — and the sixth is the file size in kilobytes. Each image collapses to six numbers before the model sees it.

The question this model answers is narrow and worth stating precisely: are the Deliverable 1 features linearly separable across 38 classes? At 0.274 macro-F1 the answer is no, but the number is also not zero, and both halves of that matter. It clears the stratified dummy by a factor of ten, so the features do carry real signal — a diseased leaf really is measurably different in average colour from a healthy one. It falls far short of anything usable, so a linear decision boundary over global colour means cannot separate 38 crop-and-disease combinations.

One implementation detail affects how the number should be read. The model is fitted on a per-class sample capped at 300 images rather than on the full 38,013, because computing the features requires opening every file at full resolution and the cost was not worth paying for a model whose role is to be beaten. Scoring is on the complete validation set, so the reported figure is not optimistic; if anything the cap slightly understates what this model could do. It does not change the conclusion — no amount of additional training data makes a six-dimensional linear model separate these classes.

The feature `file_size_kb` deserves a flag here rather than in the next section, because it enters the project at this point. File size is a property of the JPEG encoder, not of the plant: images with more high-frequency detail compress less well and produce larger files. It is a legitimate predictor in the narrow sense that it correlates with the label, and an illegitimate one in every sense that matters for a diagnostic tool. It is retained through the classical tier for continuity with Deliverable 1, and its influence is measured explicitly in B.4.

B.4 Random Forest on six global features

Features	the same six as B.3, unscaled
Model	<code>RandomForestClassifier(n_estimators=300, class_weight="balanced", n_jobs=-1, random_state=42)</code>
Validation macro-F1	0.382
Validation accuracy	0.507
Top feature importance	<code>file_size_kb</code> at 0.20
Compute	under one minute CPU on cached features
Notebook	<code>Step3_1_PlantVillage_RandomForest_Global6.ipynb</code>

This row exists to isolate one variable. It sees exactly the same six numbers per image as the logistic regression above; the only thing that changes is that a forest of 300 bagged decision trees can carve the feature space into arbitrary axis-aligned regions where a multinomial logistic regression can only draw hyperplanes. The difference between 0.274 and 0.382 is therefore attributable to non-linearity and to nothing else. Non-linearity is worth about 0.11 macro-F1 on this problem.

That figure only becomes interesting when it is set against the next one. Keeping the same Random Forest and expanding the input from six features to about 160 is worth roughly 0.52. The representation is worth about five times what the choice of algorithm is worth, and this is the single most transferable finding of the classical tier — it is the reason the project stopped tuning classical models and went to a class of model that constructs its own representation.

Scaling was not applied, and did not need to be: a decision tree splits on thresholds within a single feature, so a monotone rescaling of that feature changes nothing about which partitions are reachable. This is one of the practical reasons a forest is a good default on heterogeneous tabular features.

The feature importance is the part of this record that should be read with discomfort. `file_size_kb` carries a Gini importance of 0.20 — the largest of the six, and a fifth of the model's entire decision basis. A model that leans that heavily on JPEG compression is reading a property of the photograph rather than a property of the plant. On PlantVillage this works, because images of a given class were captured and encoded under similar conditions. On photographs taken by a grower with a phone it would carry no information at all, and might carry misleading information. It is the earliest concrete instance in this project of a shortcut feature, and it is the reason the interpretability work in B.11 was framed as a measurement rather than an illustration.

B.5 Random Forest on ~160 features — the baseline

This is the strongest result the project obtained without deep learning, and the mark the convolutional networks had to beat. It is treated at length because the gap between it and the scratch CNN is where the report's central claim about representation is settled.

Feature vector	160 dimensions — see Figure B.1
Feature image size	64 × 64 for histograms and texture; full resolution for the six global summaries
Model	RandomForestClassifier(n_estimators=300, class_weight="balanced", n_jobs=-1, random_state=42)
Fitted on	the full training split — 38,013 × 160
Scored on	the full 8,146-image validation set
Validation macro-F1	0.906
Validation accuracy	0.926
Errors	≈603 of 8,146
Feature extraction	273 s for train, 58 s for validation — cached to .npz
Forest fit	148 s on the Colab CPU runtime
Artefacts	random_forest.joblib, rf_per_class_recall.csv, rf_group_importance.csv, rf_run_config.json
Notebook	Step3_1_PlantVillage_RandomForest.ipynb

The feature vector

Each image is reduced to 160 numbers by four separate descriptors, computed as follows.

- **RGB colour histograms — 96 features.** Each of the three channels is binned into 32 equal-width bins over 0–255 and the counts are normalised to sum to one. This is the largest single block of the vector.
- **HSV colour histograms — 48 features.** The image is converted to hue-saturation-value and each channel is binned into 16 bins over 0–1. HSV is included alongside RGB because it separates *what colour* a pixel is from *how bright* it is, which is exactly the distinction a lesion on a shaded part of a leaf confounds in RGB.
- **GLCM texture — 10 features.** A grey-level co-occurrence matrix is built at 32 grey levels for two pixel offsets (1 and 2) and four angles (0°, 45°, 90°, 135°). Five Haralick properties — contrast, dissimilarity, homogeneity, energy and correlation — are read off each matrix and averaged over the four angles, which makes them approximately rotation-invariant. Two distances × five properties gives ten numbers.
- **The six global summaries — 6 features.** Carried forward unchanged from B.3 and B.4, and computed on the full-resolution image rather than the 64 × 64 copy so that `file_size_kb` remains the true file size.



Figure B.1. The 160-dimensional feature vector by group. Colour histograms account for 144 of the 160 dimensions; texture for 10; the Deliverable 1 global summaries for 6.

The histograms and the texture matrix are computed on a 64 × 64 downsample rather than the original image. This is a deliberate trade, and a defensible one for two of the three descriptors: a normalised histogram is a distribution over pixel values, and downsampling changes the sample size rather than the distribution. For the GLCM it is more consequential — co-occurrence at an offset of one pixel means something different at 64 × 64 than at 256 × 256 — but the same downsample is applied to every image, so the feature remains internally consistent and the comparison is unaffected. The saving was substantial: extraction over the full 46,159 train-plus-validation images took just under six minutes.

What the forest actually used

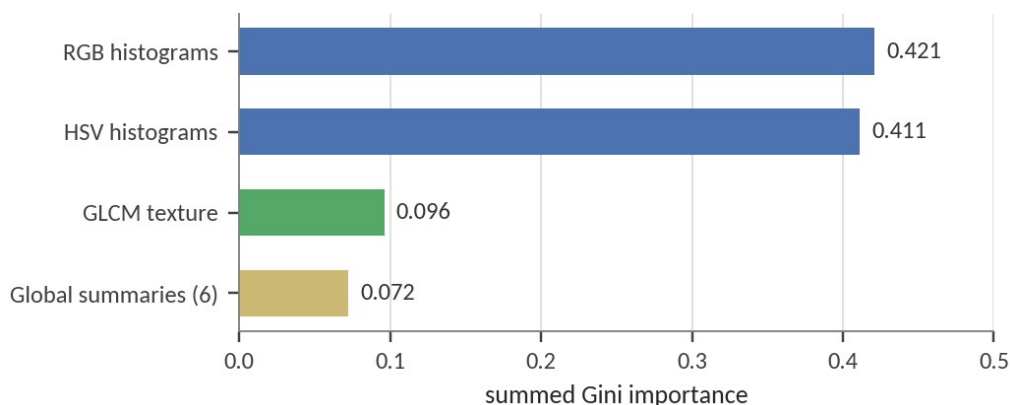


Figure B.2. Gini importance summed within each feature group. The two histogram blocks together account for 83.2% of the forest's decision basis; the six global summaries for 7.2%.

Summing the per-feature Gini importances by group gives the clearest single picture of this model: RGB histograms 0.421, HSV histograms 0.411, GLCM texture 0.096, the six global summaries 0.072. The colour histograms together carry 83% of the decision, and everything else is a rounding correction on top of them.

Two readings follow. The first is reassuring — the forest has largely stopped leaning on `'file_size_kb'`, whose entire group now accounts for 7.2% of the model where in B.4 that one feature alone accounted for 20%. Giving the model real signal displaced the shortcut, which is what one would hope. The second is the binding one: this model is, to a first approximation, a colour-histogram classifier. Its ceiling is the ceiling of that representation.

Where it fails, and why the ceiling is structural

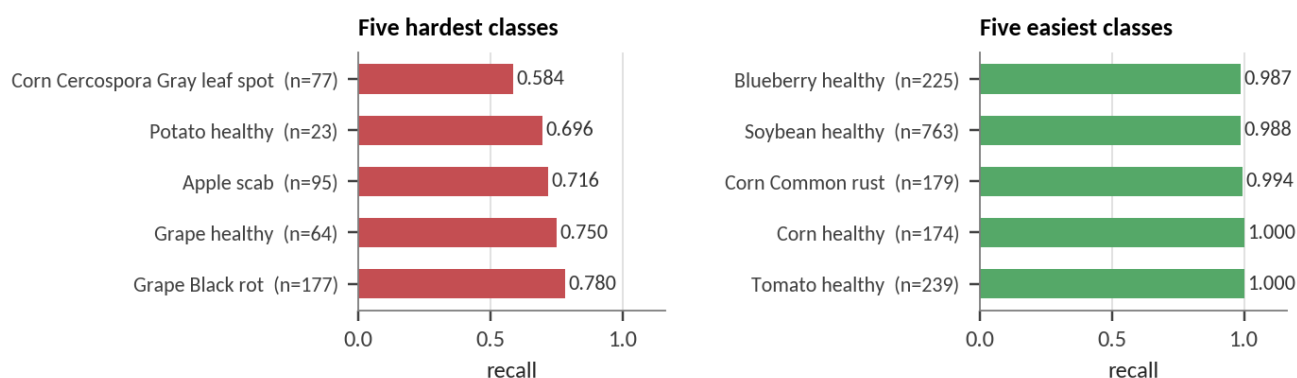


Figure B.3. Validation per-class recall, the five worst and the five best classes. The spread from 0.584 to 1.000 is the tail that macro-F1 exists to expose and that accuracy would conceal.

The per-class recalls run from 0.584 to 1.000, and the identity of the worst class is the whole story. Corn Cercospora leaf spot / Gray leaf spot is recalled at 0.584, with a precision of 0.849 — the forest finds barely more than half of them, and the images it loses go predominantly to Corn Northern Leaf Blight, which is over-predicted at 0.828 precision against 0.912 recall. Two fungal diseases on the same crop produce lesions of similar colour in different shapes, and a histogram cannot represent shape.

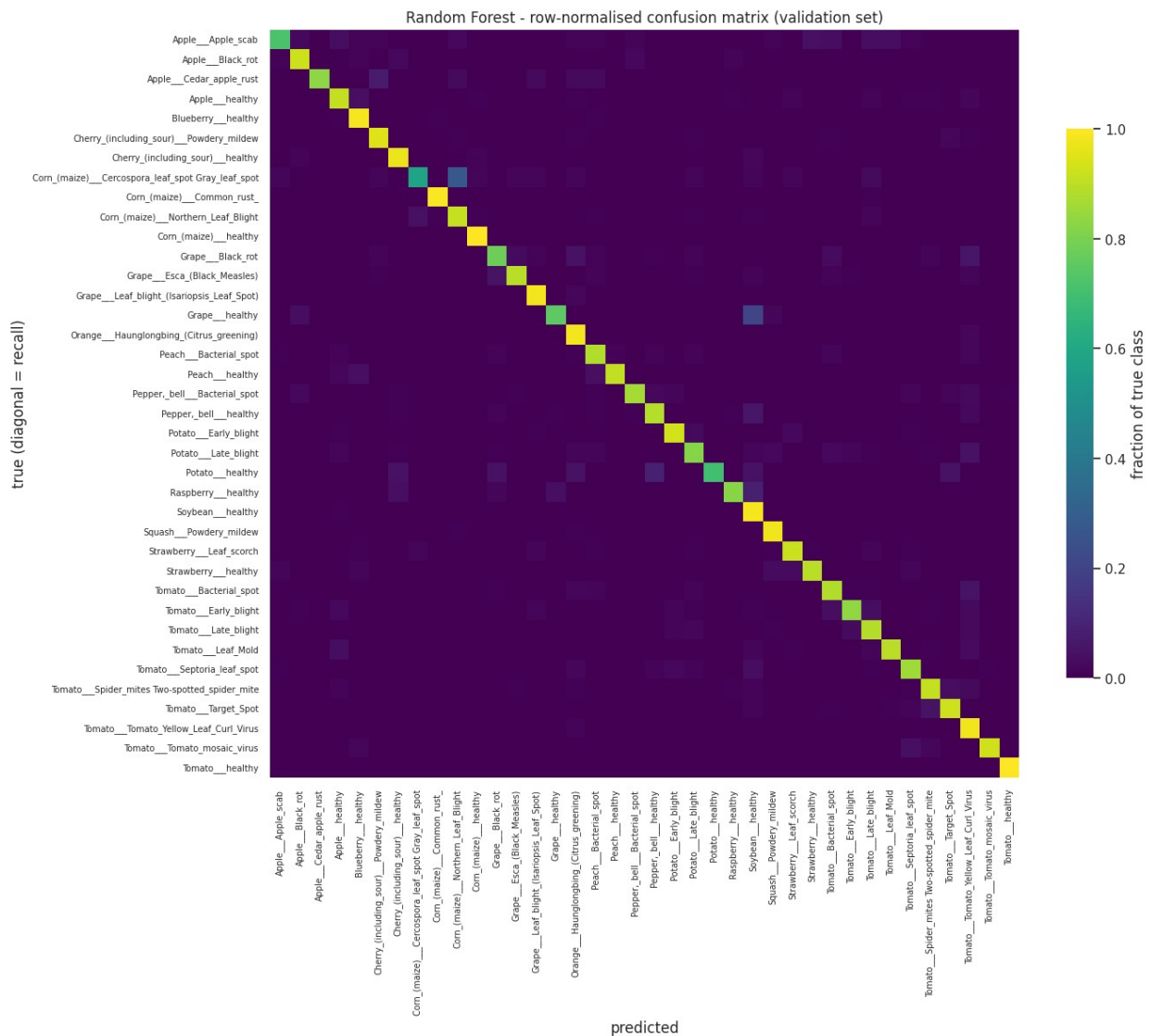


Figure B.4. Row-normalised validation confusion matrix, reproduced from the Random Forest notebook. Each row is divided by its own support, so the diagonal reads directly as per-class recall. The dark diagonal is the model working; the off-diagonal blocks are where it fails, and they cluster within crops rather than scattering across them.

The confusion matrix makes the failure structure visible in a way the recall list cannot. Errors do not scatter uniformly — they concentrate in dense off-diagonal blocks that correspond to crops, with the Corn and Grape groups the most obvious. A model reading colour histograms can tell a maize leaf from a grape leaf without difficulty, because the two differ globally in hue and shape; what it cannot do is separate two diseases *within* a crop, because those differ in the arrangement of discoloured tissue rather than in its quantity or colour.

That confusion pair is worth marking here because it recurs. It appears in this model, in the scratch CNN, in every transfer backbone, and it survives into the final model as the single class that keeps the minimum per-class recall below 0.95. It is a property of the data at 128 pixels, not a property of any particular model.

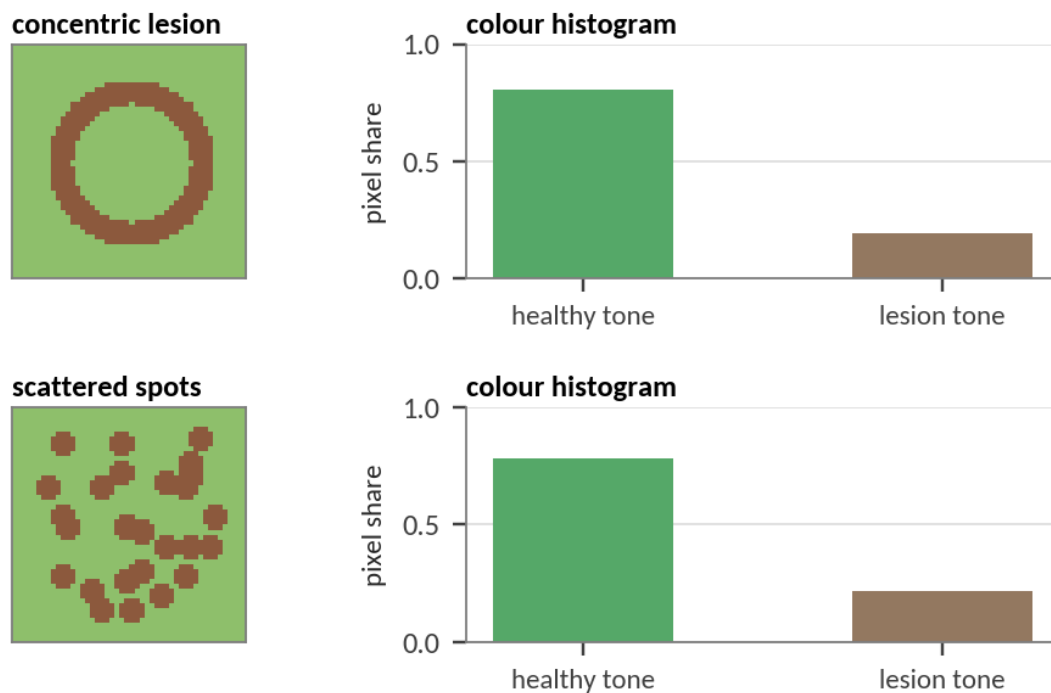


Figure B.5. Why the classical ceiling is structural. Two lesion patterns a pathologist would never confuse — a concentric ring and a scattering of spots — discolour nearly the same fraction of the leaf and therefore produce nearly the same colour histogram. What separates them is spatial, and a histogram discards it by construction.

Figure B.5 makes the point that no amount of additional trees can address. A colour histogram counts how many pixels of each colour an image contains and throws away where they were. Two diseases that discolour the same amount of tissue in different arrangements — a ring, a streak along a vein, a scattering of small spots — produce nearly identical histograms. The GLCM features recover a little of this, since co-occurrence statistics are sensitive to local structure, but they are summed over the whole image and so are themselves globally pooled. Ten texture numbers cannot describe where on a leaf a pattern occurs.

The ceiling at 0.906 is therefore a property of the representation rather than of the ensemble sitting on top of it. This is also why no boosting model was built. Gradient boosting on this same feature matrix would very likely improve on 0.906 by a few points, because sequential residual fitting extracts more from a fixed representation than bagging does. It would not close the 0.067 gap to the scratch CNN, because that gap is not about how the features are combined — it is about which features exist. Part I records this as an acknowledged gap in the comparison rather than a settled result, and it remains the cheapest outstanding item in the project: the feature matrix is already cached, so closing it is a single `'fit'` call.

How the forest is configured

Every hyper-parameter except the tree count is a scikit-learn default, and the defaults are worth stating because they are load-bearing rather than incidental.

Setting	Value	What it does here
<code>n_estimators</code>	300	trees in the ensemble; the only value set deliberately
<code>max_features</code>	"sqrt" (default)	each split considers a random 12 of the 160 features
<code>max_depth</code>	None (default)	trees grow until leaves are pure or cannot be split further
<code>min_samples_split</code>	2 (default)	no minimum-size constraint on an internal node
<code>bootstrap</code>	True (default)	each tree is grown on a resample of the 38,013 training rows
<code>class_weight</code>	"balanced"	sample weights inverse to class frequency, applied in the impurity criterion
<code>random_state</code>	42	fixes both the bootstrap draws and the per-split feature subsets

Table B.1. The forest's configuration. Only `'n_estimators'` and the class weighting were chosen; everything else is the library default, and this model was not tuned.

Two of these deserve comment. ``max_features="sqrt"`` means each split is chosen from a random twelve of the 160 columns rather than from all of them, which is what decorrelates the trees and is the mechanism by which averaging them reduces variance. With 144 of the 160 columns being histogram bins, and neighbouring bins strongly correlated, that subsampling is doing real work: without it the trees would repeatedly select from the same handful of informative bins and the ensemble would behave much like a single deep tree.

``class_weight="balanced"`` enters at the impurity calculation rather than at the sampling stage. Each training row carries a weight inverse to its class frequency, and both the Gini impurity of a candidate split and the majority vote at a leaf are computed over weighted counts. A leaf containing three healthy-Potato images and eight Orange-Huanglongbing images therefore predicts healthy Potato, because 3×9.44 exceeds 8×0.26 . This is the same correction the neural models receive through ``class_weight`` in ``fit``, arriving by a different route.

The tree count was set to 300 rather than tuned. Random Forest error falls quickly with the number of trees and then flattens; beyond a hundred or two the marginal tree changes little, and no amount of additional trees can overfit an ensemble of this construction, so the only cost of the choice is fit time. It was not searched because Section B.4 had already established that the algorithm is worth about a fifth of what the representation is worth, and tuning the ensemble was not where the remaining gain lay.

Why a Random Forest specifically

Three properties made the forest the right classical model here, and none of them is its accuracy. It accepts heterogeneous features — normalised histogram bins in $[0,1]$ alongside a file size in kilobytes alongside a mean channel value in $[0,255]$ — without any rescaling, because it splits on thresholds within single features. It accepts class weights directly, which was a hard requirement of the comparison protocol. And it reports which features it relied on, which is what made Figure B.2 possible and what turned this row from a number into a finding. An RBF support-vector machine would plausibly have scored comparably; it would have required scaling, it would have cost far more to fit on $38,013 \times 160$, and it would have told us nothing about *why*.

B.6 The scratch CNN

The first model in the project to construct its own representation, and the model that established the shape of the result: 0.973 macro-F1 from 263,654 parameters, against 0.906 from a classical pipeline that extracts 160 features per image. It is treated at length because it is the correct control against which the transfer models must be judged, and because the report's argument about representation is only as strong as the like-for-like comparison between this model and B.5.

Architecture	four convolutional blocks, global average pooling, two dense layers
Input	$128 \times 128 \times 3$, pixels in 0-255
Normalisation	Rescaling($1/255$) as the first layer of the model
Parameters	263,654 total — 262,950 trainable, 704 non-trainable
Optimiser	Adam, initial learning rate $1e-3$
Loss	sparse categorical cross-entropy
Batch size	$32 \cdot 1,188$ steps per epoch
Epoch budget	20, with EarlyStopping(monitor=val_loss, patience=4, restore_best_weights=True)
Schedule	ReduceLROnPlateau(monitor=val_loss, factor=0.5, patience=2, min_lr= $1e-5$)
Validation macro-F1	0.973 (figure of record — see the note on provenance below)
Validation accuracy	0.977
Errors	190 of 8,146
Compute	$\approx 60-85$ s per epoch on a T4; roughly 20 min for a full run
Artefacts	baseline_cnn.keras, baseline_results.csv, baseline_per_class_recall.csv, baseline_run_config.json
Notebook	Step3_1_PlantVillage_Baseline_Models.ipynb

The architecture, layer by layer

	<i>output shape</i>	<i>parameters</i>
Input	$128 \times 128 \times 3$	—
Rescaling $1/255$	$128 \times 128 \times 3$	—
Augment (train only)	$128 \times 128 \times 3$	—
Conv 3×3 , 32 • BN • ReLU	$128 \times 128 \times 32$	1,024
MaxPool 2×2	$64 \times 64 \times 32$	—
Conv 3×3 , 64 • BN • ReLU	$64 \times 64 \times 64$	18,752
MaxPool 2×2	$32 \times 32 \times 64$	—
Conv 3×3 , 128 • BN • ReLU	$32 \times 32 \times 128$	74,368
MaxPool 2×2	$16 \times 16 \times 128$	—
Conv 3×3 , 128 • BN • ReLU	$16 \times 16 \times 128$	148,096
MaxPool 2×2	$8 \times 8 \times 128$	—
GlobalAveragePooling2D	128	—
Dropout 0.3	128	—
Dense 128 • ReLU	128	16,512
Dense 38 • softmax	38	4,902
263,654 total • 262,950 trainable • 704 non-trainable (BatchNorm statistics)		

Figure B.6. The scratch CNN in full. Shaded rows carry parameters. Normalisation and augmentation are layers inside the model rather than dataset operations, so the saved checkpoint carries its own preprocessing and augmentation is structurally inactive at inference.

The design is deliberately unremarkable: four repetitions of convolution, batch normalisation, ReLU and 2×2 max pooling, with the filter count doubling at each of the first three steps and holding at 128 for the fourth. All convolutions are 3×3 with `padding="same"`, so the spatial size is controlled entirely by the pooling schedule and halves four times, $128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8$.

Two choices in the head are worth naming. Global average pooling rather than flattening collapses the final $8 \times 8 \times 128$ volume to a 128-vector by averaging each channel over space. A `Flatten` at that point would have produced 8,192 features and a first dense layer of over a million parameters — four times the entire network as built — for no gain, since the spatial arrangement at 8×8 has already served its purpose. Dropout at 0.3 sits between the pooling and the 128-unit dense layer, which is the only place in the network where a large number of parameters meets a small number of activations and therefore the only place where memorisation is a realistic risk.

Batch normalisation is placed after each convolution and before the pooling. Its 704 non-trainable parameters are the running means and variances of the four normalised layers, $2 \times (32 + 64 + 128 + 128)$; the matching 704 scale and offset parameters are trainable and are counted in the 262,950. This detail is minor here and becomes central in B.11, where the decision to freeze exactly these statistics is what made fine-tuning stable.

Where the parameters and the arithmetic sit

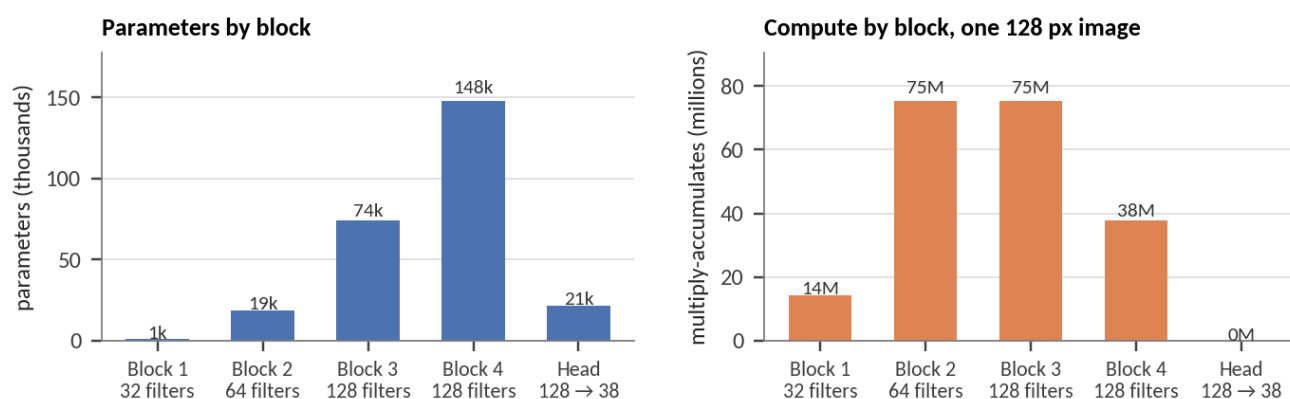


Figure B.7. Parameters and multiply-accumulates by block, for one 128-pixel image. The two distributions run in opposite directions: the deepest block holds the most parameters and does the least work, because each of its filters is applied at 16×16 rather than at 128×128 .

Figure B.7 shows a property of convolutional networks that is easy to state and easy to forget. Parameters accumulate towards the back — block 4 holds 148,096 of the network's 263,654 — while arithmetic accumulates towards the front, because a filter in block 1 is applied at 128×128 positions and a filter in block 4 at only 16×16 . One forward pass costs about 203 million multiply-accumulates in total, of which blocks 2 and 3 carry three quarters.

This matters for the comparison in B.11. DenseNet-121 has 27 times this network's parameters but performs only about four and a half times the arithmetic on the same 128-pixel input, because almost all of its parameters live in deep blocks operating at 8×8 and 4×4 . Parameter count and compute cost are not interchangeable measures of model size, and the argument about which model is "heavier" depends on which one is meant.

Receptive field — why four blocks and not two

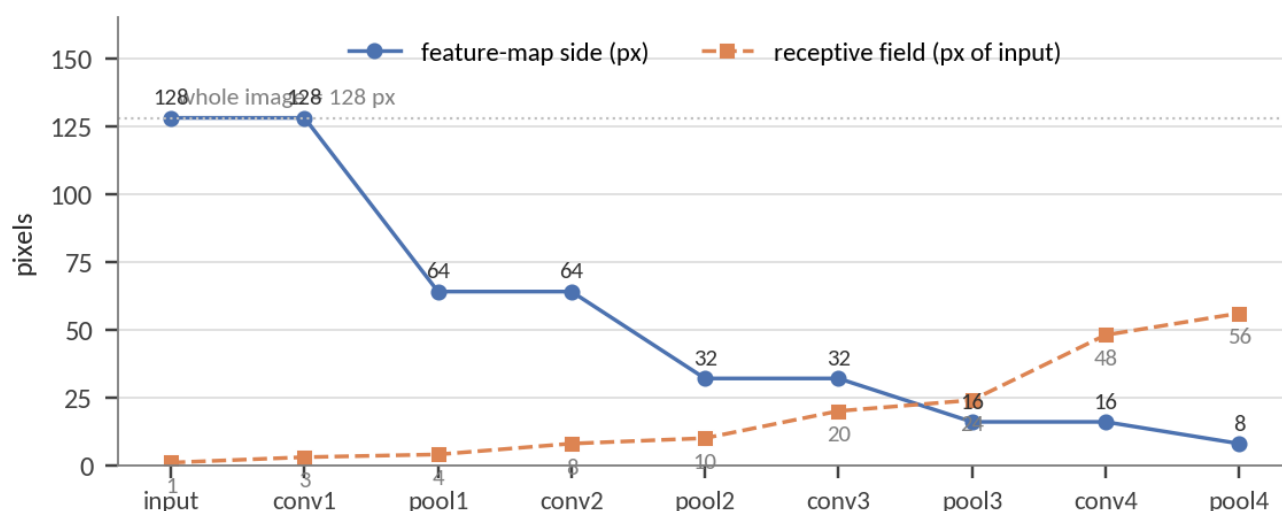


Figure B.8. Feature-map size and receptive field through the network. By the fourth convolution a single unit responds to a 48-pixel region of a 128-pixel image — large enough to contain a whole lesion — while the map is small enough that the global average pooling that follows is summarising features rather than pixels.

A single 3×3 convolution sees nine pixels. Stacking convolutions and pooling between them enlarges the region of the input that any one deep unit responds to, and Figure B.8 tracks that growth: 3 pixels after the first convolution, 8 after the second, 20 after the third, 48 after the fourth. The fourth block's units therefore span a little over a third of the image in each direction, which is the scale at which a lesion, rather than an edge or a patch of texture, becomes visible as an object.

That is the design justification for four blocks. Two would leave the deepest units with a receptive field of roughly eight pixels — enough to detect discoloured texture, not enough to distinguish a ring from a scattering, which is precisely the distinction Figure B.5 showed the classical model could not make. The 0.067 macro-F1 the scratch CNN gains over the Random Forest is bought here, in the ability to represent where a discoloured pixel sits relative to its neighbours.

Training dynamics

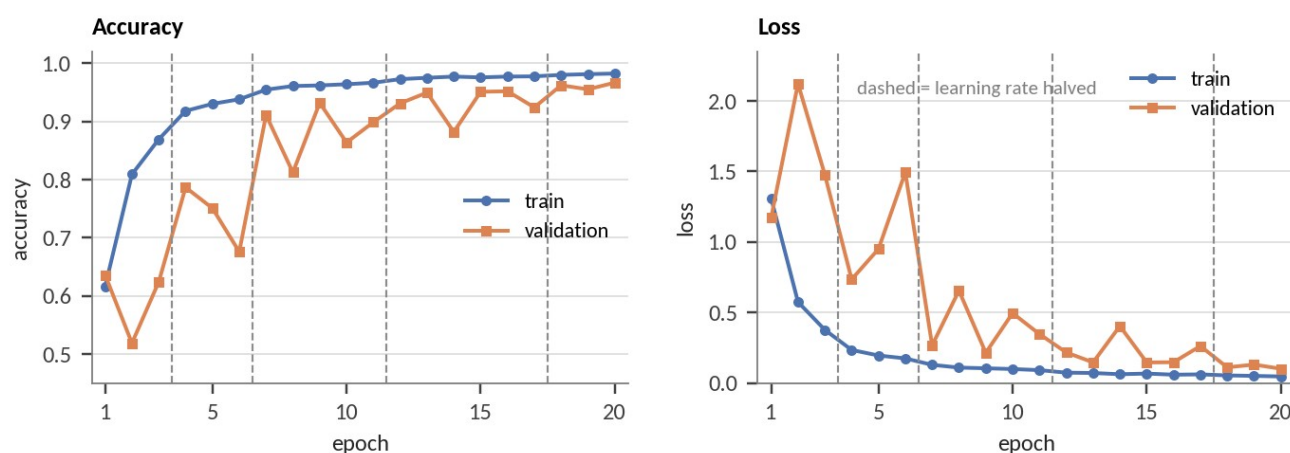


Figure B.9. Twenty epochs of the scratch CNN, from the seed-reproduced run — the only run of this architecture whose full epoch log survives. Dashed lines mark the four occasions on which ReduceLROnPlateau halved the learning rate.

The training curve is instructive and not entirely comfortable. Training accuracy climbs smoothly from 0.62 to 0.98 and never wavers. Validation accuracy does something quite different: it reaches 0.64 in the first epoch, falls to 0.52 in the second, and oscillates violently for the first six epochs — 0.79, 0.75, 0.68 — before settling. Validation loss behaves worse still, rising from 1.17 to 2.12 between epochs one and two while training loss falls by more than half.

This is not over-fitting, which would show as a slowly widening gap rather than as oscillation. It is a learning rate that is too large for the loss surface at the point the model has reached, so each epoch overshoots in a different direction and

the validation set lands wherever the last few batches left the weights. The diagnosis is confirmed by what fixes it: `ReduceLROnPlateau` halves the rate at epochs 4, 7, 12 and 18, and the curve steadies almost immediately after the second reduction — validation accuracy reaches 0.91 at epoch 7, the first epoch at $2.5e-4$, and never returns to the earlier instability.

Three things follow, and each is used elsewhere in this appendix. The first is that the learning-rate schedule, not the architecture, is doing most of the stabilising here — which is a reason to be sceptical of the tuned CNN's finding in B.7 that a lower initial learning rate helps, since the schedule already provides one adaptively. The second is that the train-validation gap at convergence is small — 0.982 against 0.967 in this run — so there is no over-fitting to correct and the augmentation and dropout in place are sufficient. The third is methodological: this is exactly the kind of curve that a BatchNorm statistics drift can also produce, and distinguishing the two is the diagnostic problem described in B.11.8.

The epoch budget of 20 was never reached by early stopping in this run, which means the model was still improving slowly when the budget ran out. A longer budget would probably have added a little, and the hyper-parameter search in B.7 did not test it — epochs were not among the five searched dimensions.

A note on provenance

Three validation figures exist for this architecture and it is worth recording which is which, since Part I's Table 1 carries only one of them.

Run	Val macro-F1	Val acc.	Status
Original Step 3.1 session	0.979	0.982	checkpoint lost to a runtime recycle
Checkpoint used in Deliverable 2	0.973	0.977	figure of record in Table 1
Seed-reproduced test-set run	0.960	0.967	used only for the validation-to-test gap check

Table B.2. Three runs of the same architecture and recipe. The spread of 0.019 between the first and third is consistent with the ± 0.02 run-to-run variation documented for cloud GPUs in Section A.1.

The figure of record is 0.973, because that is the score of the checkpoint that actually exists on Drive, that was used in the Grad-CAM comparison, and that produced the 190 validation errors quoted throughout. The 0.979 belongs to a session whose checkpoint did not survive; where an earlier document quotes it, the surviving artefact takes precedence.

The third row is a separate exercise and should not be read as a fourth model. It reproduces the Step 3.1 recipe from seed in a fresh runtime purely to check that validation and test behave alike, and it returned validation 0.960 against test 0.961 — a gap of -0.002 . The absolute level is lower than the other two runs for the same non-determinism reason, but the *gap* is the quantity that run was built to measure, and it is unaffected.

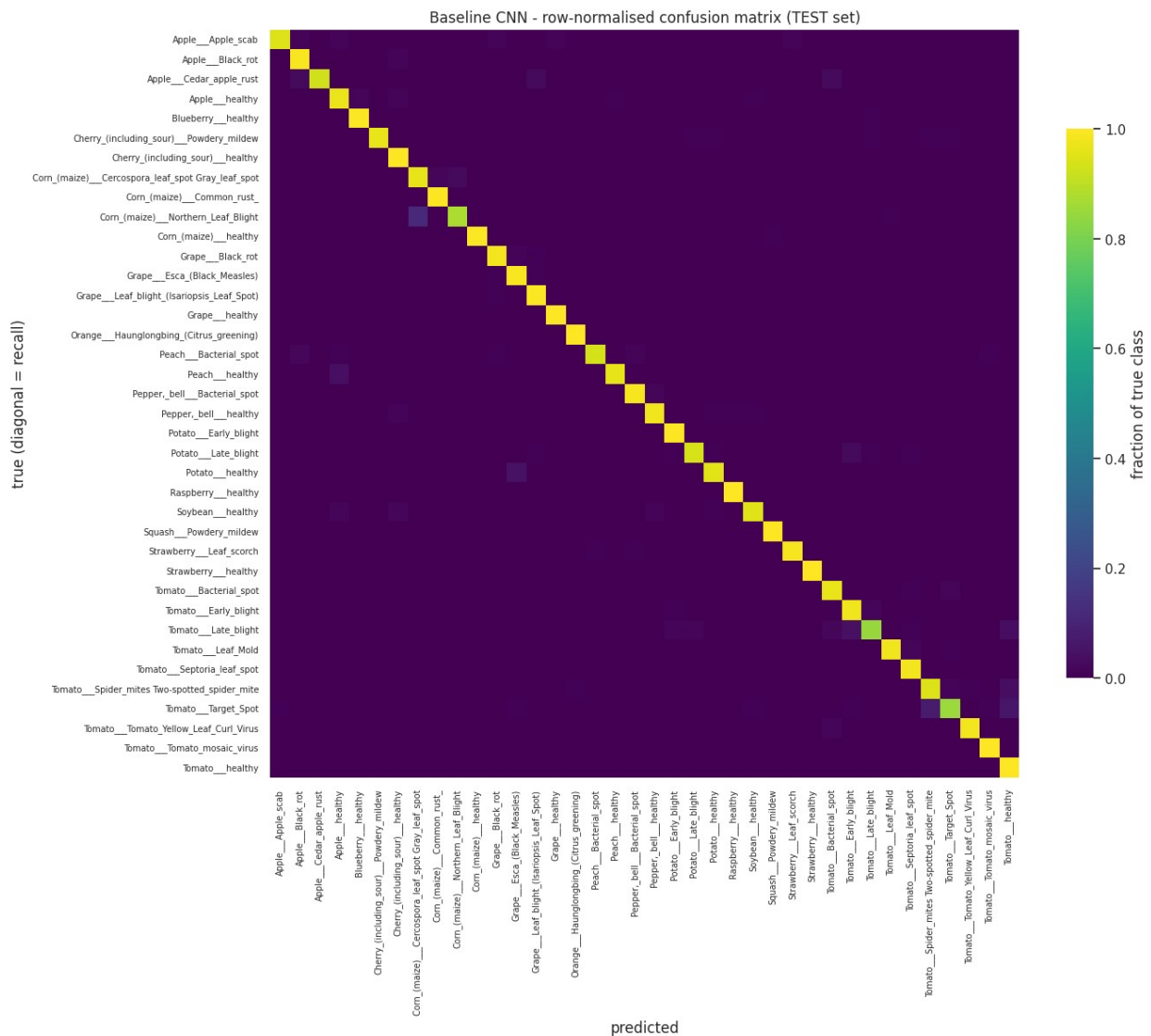


Figure B.10. Row-normalised test-set confusion matrix for the scratch CNN, reproduced from the evaluation notebook. This is the third run in Table B.2 — the seed-reproduced one scoring 0.960 on validation and 0.961 on test — and not the 0.973 checkpoint of record. It is included because it is the only surviving confusion matrix for this architecture, and because the structure of the errors is the informative part rather than their absolute count.

Set beside the Random Forest's matrix, the change is not a uniform lightening of the off-diagonal. The dense within-crop blocks have largely dissolved — Grape and Corn no longer bleed across their own diseases at anything like the earlier rate — while a handful of specific pairs survive, notably within Tomato. That is the signature of a model that has acquired spatial representation: the confusions that depended on shape being invisible are gone, and what remains are cases where the shapes themselves are genuinely alike. The same residue, further thinned, is what appears in the final model's five-pair error budget.

B.7 The tuned CNN

Purpose	to establish whether the hand-chosen baseline configuration had headroom
Search	keras-tuner BayesianOptimization, 15 trials, objective val_accuracy, seed 42
Search space	learning_rate $\in [1e-4, 3e-3]$ log · dropout $\in [0.2, 0.5]$ step 0.1 · dense_units $\in \{128, 256\}$ · base_filters $\in \{24, 32, 48\}$ · aug_strength $\in \{0.05, 0.10, 0.15\}$
Trial protocol	ranked on a stratified sub-sample of ≤ 250 training images per class; winner retrained on the full 38,013
Result	the search returned the baseline configuration on four of five knobs; only the learning rate moved, from $1e-3$ to $\approx 3.3e-4$
Validation macro-F1	0.970
Validation accuracy	0.973
Compute	≈ 95 min on a T4 for the search, plus one full retrain
Artefacts	tuned_cnn.keras, tuned_best_hp.json, tuning_top_trials.csv, baseline_vs_tuned.csv
Notebook	Step3_2_PlantVillage_CNN_Tuning.ipynb

Bayesian optimisation was chosen over an exhaustive grid because it fits a surrogate model to the trials it has already run and spends its remaining budget where that surrogate predicts improvement. With fifteen trials over five dimensions — a grid of the same resolution would need several hundred — that concentration is the only thing that makes the search affordable. Batch size and input resolution were deliberately held fixed rather than searched, because both were fixed by the comparison protocol and a model that won by changing them would not have been admissible in Table 1.

The result is a negative one and is reported as such. The tuned model scores 0.970 against the baseline's 0.973, a difference comfortably inside the ± 0.02 run-to-run band, so the honest reading is that the two configurations are indistinguishable. The single hyper-parameter the search did move — a lower learning rate — is most plausibly an artefact of the ranking protocol: trials were scored on a sub-sample of at most 250 images per class, roughly a quarter of the training data, and the optimal learning rate is not scale-invariant to dataset size. A rate that converges well on 9,500 images is not necessarily the rate that converges well on 38,013.

The value of the experiment is what it ruled out. It says the residual errors at this level are not caused by mis-set dials, and therefore that further progress had to come from a different architecture rather than from better settings of this one. That conclusion is what sent the project to transfer learning, and without it the move would have been a guess.

B.8 Transfer — MobileNetV2

Backbone	MobileNetV2, ImageNet weights, include_top=False — 154 layers, 2,257,984 parameters
Input	128 × 128 × 3, pixels in 0–255
Preprocessing	mobilenet_v2.preprocess_input inside the model, after augmentation
Head	GlobalAveragePooling2D → Dropout → Dense(38, softmax); 48,678 parameters
Final feature grid	4 × 4 × 1280
Phase 1	backbone frozen, LR 1e-3, up to 12 epochs
Phase 2	backbone unfrozen, LR 1e-5, up to 10 epochs
Validation macro-F1	0.919
Validation accuracy	0.932
Worst class	Tomato Early blight at 0.580 recall
Compute	≈40–85 s per epoch on a T4
Notebook	Step3_2_PlantVillage_Transfer_Learning.ipynb

The cheapest possible test of whether pretrained features help at all. MobileNetV2 is designed for phones — depthwise-separable convolutions and inverted residual bottlenecks keep it under 2.3 million parameters — so a run costs little and the answer arrives quickly. The answer was that at 0.919 it did not beat a 264-thousand-parameter network trained from scratch, and the per-class detail is worse than the aggregate: Tomato Early blight at 0.580 recall is a class the scratch CNN handled without difficulty.

Two causes are separable and both are instructive. The first is the fine-tuning protocol. This run unfreezes the entire backbone in phase 2, including every BatchNorm layer, which lets the normalisation statistics drift on batches of 32 while the weights beneath them are also moving. That is the configuration the project later abandoned; the corrected protocol described in B.11 was developed in response to exactly this instability. A later rerun under the corrected protocol reached 0.958 at 128 pixels and 0.961 at 256, both still below the scratch CNN, but its notebook was lost to a runtime recycle and only the surviving 0.919 artefact appears in Table 1. Part I flags this.

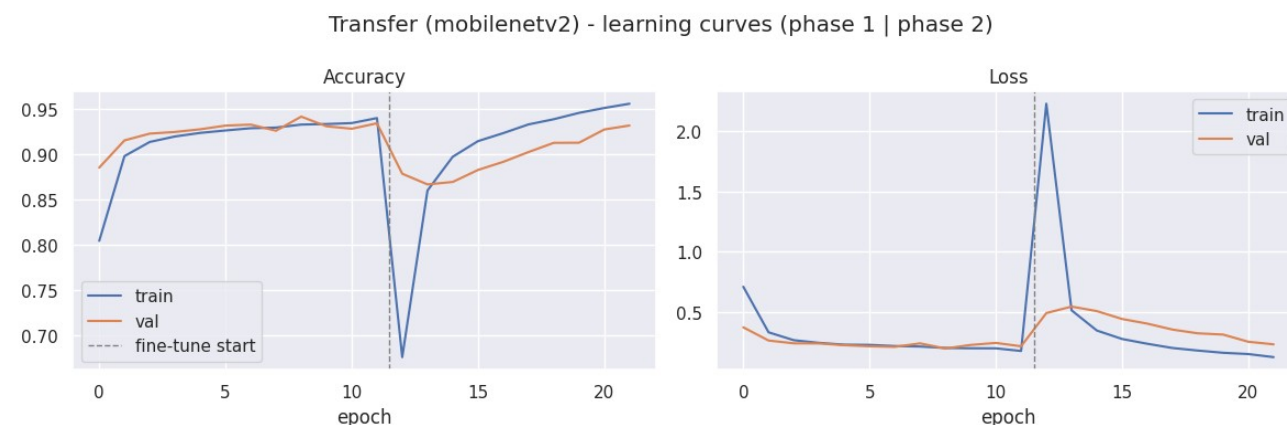


Figure B.11. Learning curves for the MobileNetV2 run, reproduced unedited from ‘Step3_2_PlantVillage_Transfer_Learning.ipynb’. The dashed line marks the point at which the backbone was unfrozen and the learning rate dropped from 1e-3 to 1e-5. The discontinuity there, and the instability that follows it, is what the corrected protocol described in B.11.8 was written to fix. The DenseNet-121 notebook used this same plotting code, so the final model’s curves take this form — but the data here is MobileNetV2’s.

The second is resolution collapse, and it is the more interesting of the two. MobileNetV2 downsamples by a factor of 32, so a 128-pixel input arrives at the classifier as a 4 × 4 grid — sixteen spatial positions from which to distinguish thirty-eight fine-grained classes. The backbone was designed for 224-pixel input, where the same architecture yields 7 × 7. The comparison in Table 1 is fair at fixed compute and unfair to the architecture as its authors intended it, and Part I lists this among its limitations.

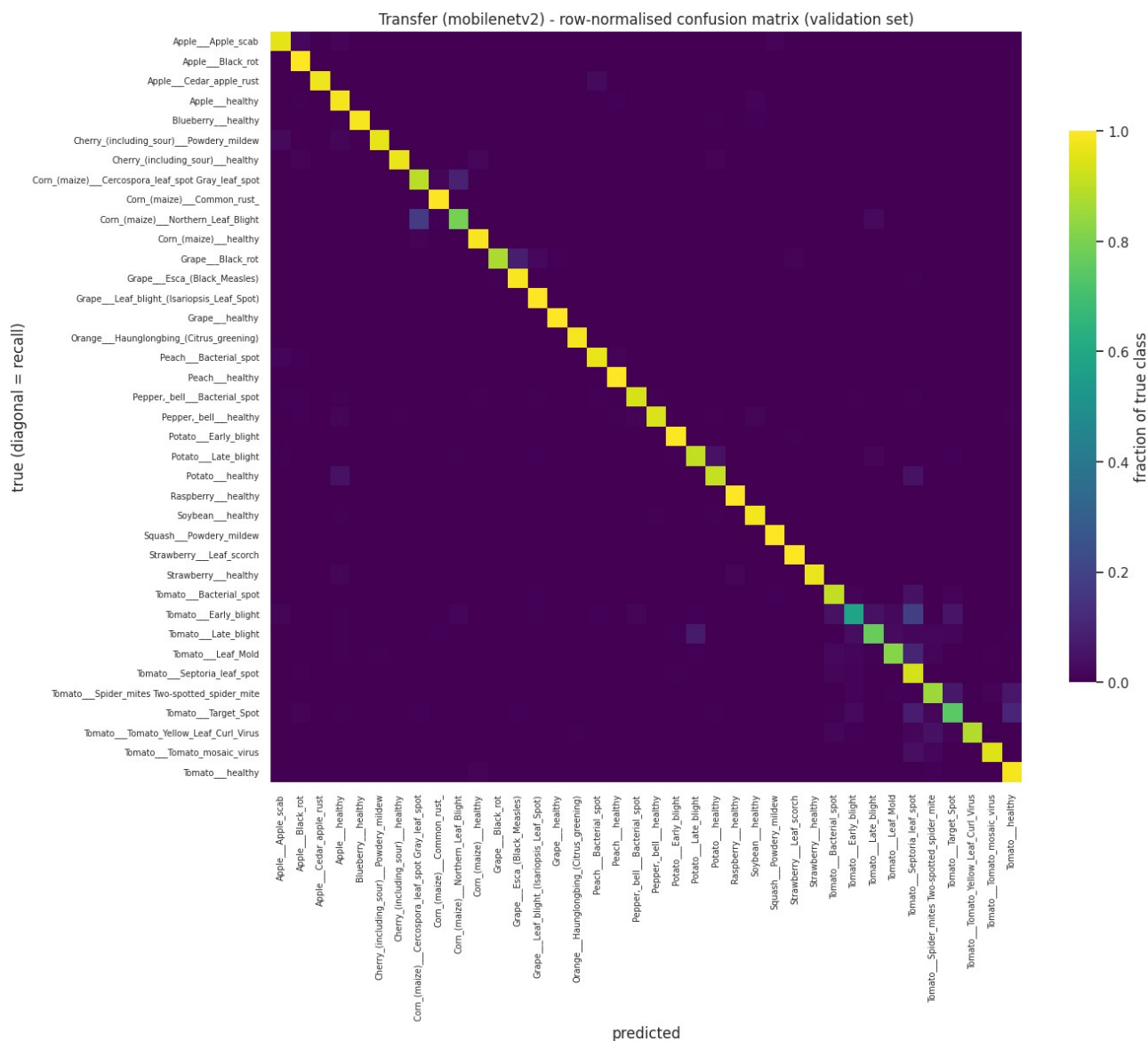


Figure B.12. Row-normalised validation confusion matrix for MobileNetV2, reproduced from the run notebook. The diagonal reads as per-class recall; the dark cell is Tomato Early blight at 0.580, a class the scratch CNN handled without difficulty. Best read at screen zoom — a 38 × 38 matrix cannot be made legible at page size.

B.9 Transfer — ResNet50

Backbone	ResNet50, ImageNet weights, include_top=False — ≈23.6 million parameters
Input	224 × 224 × 3
Head	GlobalAveragePooling2D → Dropout → Dense(38, softmax); 77,862 parameters
Protocol	corrected — top ≈30% of the backbone unfrozen, all BatchNorm frozen, base called with training=False, one ModelCheckpoint across both phases
Fine-tune assertion	trainable parameters rose 77 K → 17.26 M
Validation macro-F1	0.978
Validation accuracy	0.981
Compute	≈65 min on a T4
Notebook	rebuilt as Step3_2_PlantVillage_ResNet50.ipynb; run by Scheima

The reference architecture, included because most published results on this dataset are reported against it. Its record is short but its history is the most instructive in the transfer tier, because the first version of this run failed twice over and neither failure raised an error.

The first fault was the split. That run built its own partition with seed 123 rather than loading the canonical CSVs, which put roughly 5,700 of the sealed test images into its training data. Its reported score was therefore not comparable to anything else in Table 1 and was not a held-out score at all. The second fault was subtler: the code set ``base_model.trainable = False`` after the per-layer unfreeze loop, and under Keras 3 the model-level flag overrides the layer-level ones, so phase 2 trained zero parameters. The run completed, printed a plausible-looking number, and had not fine-tuned anything.

Both faults are now guarded in code — the split fingerprint gate described in A.2, and the trainable-parameter assertion described in B.11.9. The rerun on the canonical split with the corrected protocol returned 0.978, which clears the scratch CNN by 0.005: inside the noise band, and reported as a match rather than a win.

B.10 Transfer — EfficientNetB0

Backbone	EfficientNetB0, ImageNet weights, include_top=False — ≈4.10 million parameters
Input	224 × 224 × 3
Head	GlobalAveragePooling2D → Dropout → Dense(38, softmax); 48,678 parameters
Protocol	corrected — top ≈30% unfrozen, BatchNorm frozen, global-best checkpoint across both phases
Validation macro-F1	0.982
Validation accuracy	0.984
Worst class	Tomato Spider mites at 0.861 recall
Compute	≈60 min on a T4
Notebook	run by Adrian

EfficientNetB0 represents a family designed by scaling depth, width and input resolution together under a single compound coefficient rather than one at a time. On aggregate it is the second-strongest model in the project at 0.982, and on aggregate alone the case for preferring it over the scratch CNN would be weak — the 0.003 margin is nowhere near the noise band.

Where it separates from the final model is in the tail, and not in its favour. Its minimum per-class recall is 0.861 on Tomato Spider mites against DenseNet-121's 0.946, and Tomato Early blight sits at 0.913 against DenseNet's 0.980. A related weakness shows in its precision: Tomato Target Spot is recalled at 0.976 but with precision 0.798, meaning the model over-predicts that class — almost certainly absorbing spider-mite stippling into it, which is the same confusion that survives as the largest single pair in the final model's error budget.

One caveat applies to this row and to B.9. Both were fine-tuned at 224 pixels rather than 128, which is a deviation from the fixed-resolution protocol. It was documented at the time and judged to be a minor confound on the evidence available — ResNet50 at 224 still landed below DenseNet-121 at 128 — but it means neither row is a strict backbone-only swap, and Part I lists it as a limitation rather than resolving it.

B.11 DenseNet-121 — the final model

This is the model the project selected, evaluated once against the sealed test partition, and is claiming to understand. Its record is the longest in this appendix because every claim in Sections 9 to 11 rests on something recorded here.

Backbone	DenseNet-121, ImageNet weights, include_top=False — 427 layers, 7,037,504 parameters
Head	GlobalAveragePooling2D → Dropout → Dense(38, softmax); 38,950 parameters
Total parameters	7,076,454
Input	128 × 128 × 3, pixels in 0–255
Preprocessing	densenet.preprocess_input inside the model, after augmentation
Batch size	32
Phase 1	backbone frozen; Adam at LR 1e-3; 5 fixed epochs (1–5 of 24); 38,950 trainable parameters
Phase 2	306 of 427 layers unfrozen, all 121 BatchNorm layers frozen; Adam at LR 1e-5; 19 epochs (6–24 overall), early stop at phase-2 epoch 19 with best weights from phase-2 epoch 15; 6,909,158 trainable parameters
Class weights	balanced, verified as passed to both fit calls
Validation macro-F1	0.9918 · accuracy 0.9931 · 56 errors of 8,146
Test macro-F1	0.9919 95% CI [0.9896, 0.9941] · accuracy 0.9930 · 57 errors of 8,146
Minimum per-class recall	0.9481 on both validation and test
Training time	72 minutes on a T4
Checkpoint	densenet121.keras · MD5 afe3478450c41d91a6ac1e57a2ada44d
Split	SPLIT_ID 9e33ec57c1ec, verified at startup
Notebook	09_transfer_densenet121.ipynb · sealed evaluation: 12_final_test_evaluation.ipynb

B.11.1 Why this backbone, among the four

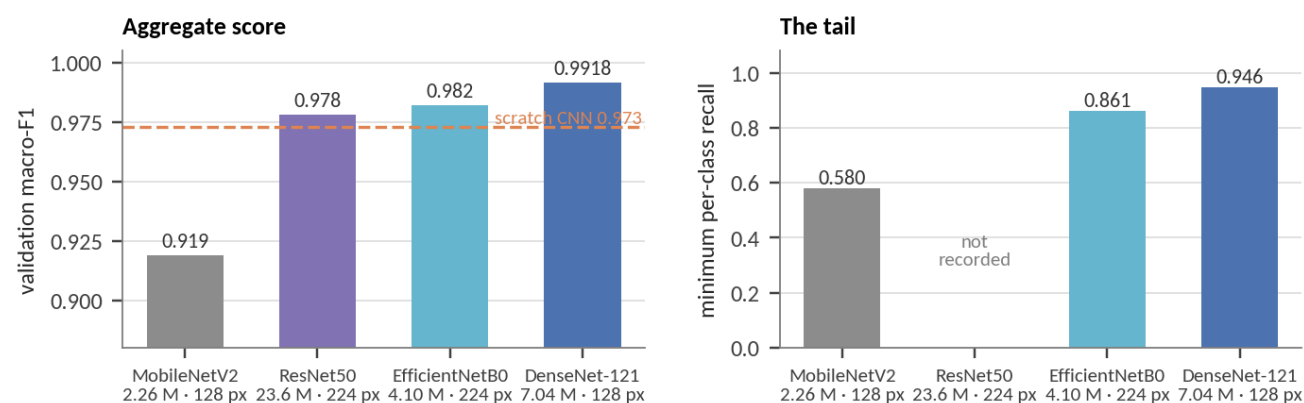


Figure B.13. The four transfer backbones on the shared validation set. Left: aggregate macro-F1, with the scratch CNN marked. Right: the lowest per-class recall each model achieved. ResNet50's per-class figures were not recorded in a surviving artefact.

Four ImageNet backbones were fine-tuned on the canonical split, chosen to span the design space rather than to be individually plausible winners: MobileNetV2 as the lightweight option built for phones, ResNet50 as the long-standing reference, EfficientNetB0 as a representative of compound scaling, and DenseNet-121 for its feature reuse. Between them they cover four points on the size-versus-capacity trade-off, from 2.3 to 23.6 million parameters.

Backbone	Params	Input	Val macro-F1	Min class recall	Notes
MobileNetV2	2.26 M	128 px	0.919	0.580	unstable full-backbone fine-tune
ResNet50	23.6 M	224 px	0.978	not recorded	corrected rerun on canonical split
EfficientNetB0	4.10 M	224 px	0.982	0.861	over-predicts Tomato Target Spot
DenseNet-121	7.04 M	128 px	0.9918	0.946	the final model

Table B.3. The transfer tier compared. Parameter counts are backbone only. Two rows were run at 224 pixels, which is a documented deviation from the fixed-resolution protocol and is listed among Part I's limitations.

The aggregate scores separate DenseNet-121 from the field by less than they appear to. Its 0.9918 against EfficientNetB0's 0.982 is a margin of 0.010, half the documented ± 0.02 session variation, and on that number alone the two would have to be called a tie. The separation that matters is in the right-hand panel of Figure B.13. EfficientNetB0's worst class is recalled at 0.861 and MobileNetV2's at 0.580, against DenseNet-121's 0.946. On the single class that most exercises the metric this project chose, the final model is not marginally better but categorically so.

A second observation runs against the obvious ordering. ResNet50 carries more than three times DenseNet-121's parameters and was fine-tuned at nearly twice the resolution, and it still scores lower. Capacity and input size were not the binding constraints in this tier; the fine-tuning protocol was, and once it was corrected the architecture that reuses features most aggressively came out ahead at the smallest input in the group. That is the empirical case for the choice, and it is stronger than the headline margin.

B.11.2 Dense connectivity

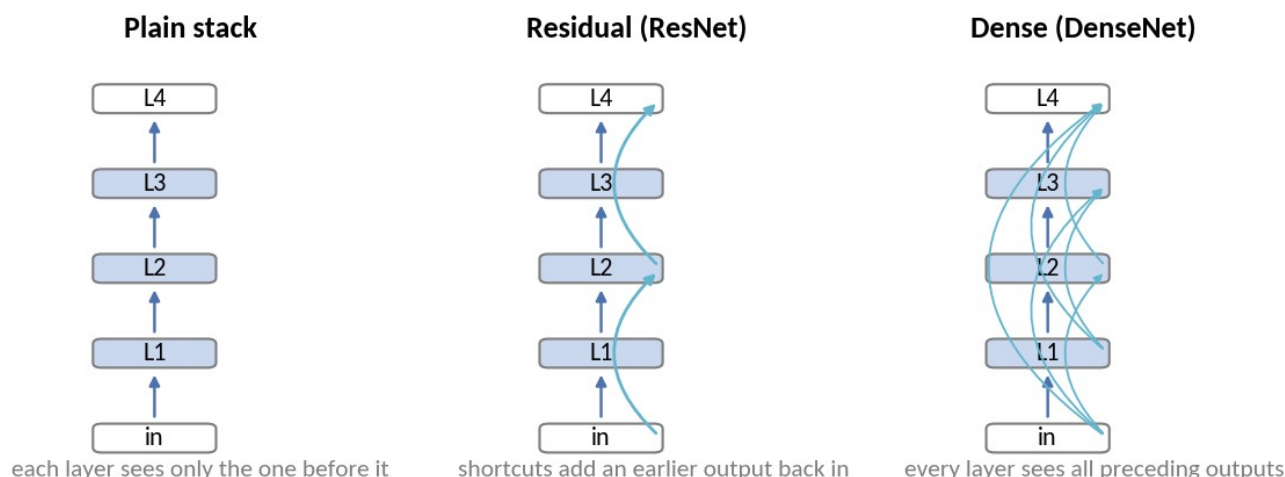


Figure B.14. Three ways of wiring a block. A plain stack passes one tensor forward. A residual network adds an earlier output back into a later one. A dense block concatenates every preceding output onto the input of every later layer, so a block of n layers has $n(n+1)/2$ connections rather than n .

DenseNet's one structural idea is that inside a block, every layer receives the concatenated outputs of all layers before it. In a plain stack, layer four sees only layer three's output; in a dense block it sees the outputs of layers one, two and three side by side, along with the block's input. The consequences run in several directions at once and it is worth separating them.

Feature reuse instead of feature relearning. In a plain network, if an early layer computes something a late layer needs, the intervening layers must carry it forward, which costs capacity. Concatenation removes that obligation: an early feature remains directly addressable at every later depth. This is why DenseNet reaches 121 layers on seven million parameters where a plain network of comparable depth needs several times as many.

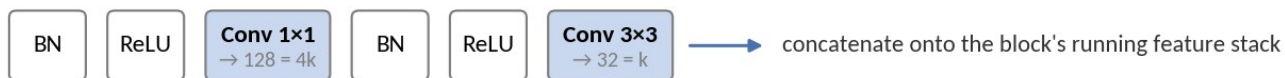
Short gradient paths. Every layer has a direct connection to the block's loss-facing output, so gradients reach early layers without passing through a long chain of multiplications. This is the same problem residual connections solve, addressed differently.

Narrow layers. Because each layer only has to contribute *new* information rather than a complete representation, it can be very thin. DenseNet-121 adds 32 channels per layer — the growth rate k — where a plain architecture at that depth would use hundreds.

Concatenation is not free. The cost of dense connectivity is memory rather than parameters. Every intermediate feature map must be retained for the whole block because later layers will read it, so activation memory during training grows quadratically in block depth. On a T4 with 16 GB this is comfortable at 128 pixels and batch 32, and it is the reason the same recipe at 256 pixels would have needed a smaller batch.

B.11.3 Inside a dense block

Dense layer (bottleneck)



Transition layer (compression $\theta = 0.5$)

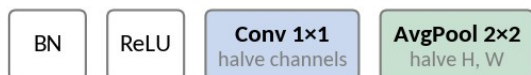


Figure B.15. The two repeating units of DenseNet-121. A dense layer contributes $k = 32$ new channels through a 1×1 bottleneck that first compresses the concatenated input to $4k = 128$. A transition layer halves both the channel count and the spatial size, which is what keeps the concatenation from running away.

A dense layer is $\text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv } 1 \times 1 \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv } 3 \times 3$, and the 1×1 convolution is doing essential work. By the twenty-fourth layer of block 3 the concatenated input is over a thousand channels deep; applying a 3×3 convolution to that directly would cost roughly $9 \times 1024 \times 32$ parameters. The bottleneck first compresses to $4k = 128$ channels with a 1×1 , then applies the 3×3 to that, which reduces the parameter cost of a deep dense layer by roughly an order of magnitude. The layer's output — 32 channels — is concatenated onto the running stack rather than replacing it.

Transition layers sit between blocks and do two things at once: a 1×1 convolution halves the channel count, which is the compression factor $\theta = 0.5$ from the original paper, and a 2×2 average pooling halves the spatial dimensions. Without them the channel count would grow monotonically across the whole network. Note that transitions use average pooling rather than the max pooling of the scratch CNN, which is consistent with the architecture's general preference for preserving rather than selecting information.

Note also that the normalisation precedes the convolution rather than following it. This pre-activation ordering matters for the freezing decision in B.11.7: a frozen BatchNorm in DenseNet is fixing the input distribution to a convolution, not rescaling its output.

B.11.4 The network at 128 pixels

	output shape	parameters
Input	$128 \times 128 \times 3$	
Conv 7×7, stride 2 · BN · ReLU	$64 \times 64 \times 64$	9,472
MaxPool 3×3, stride 2	$32 \times 32 \times 64$	—
Dense Block 1 — 6 layers	$32 \times 32 \times 256$	0.34 M
Transition 1	$16 \times 16 \times 128$	33 K
Dense Block 2 — 12 layers	$16 \times 16 \times 512$	0.93 M
Transition 2	$8 \times 8 \times 256$	132 K
Dense Block 3 — 24 layers	$8 \times 8 \times 1024$	2.71 M
Transition 3	$4 \times 4 \times 512$	526 K
Dense Block 4 — 16 layers	$4 \times 4 \times 1024$	2.06 M
Final BN · ReLU	$4 \times 4 \times 1024$	4,096
GlobalAveragePooling2D	1024	—
Dropout	1024	—
Dense 38 · softmax (our head)	38	38,950
growth rate $k = 32$ · backbone 7,037,504 + head 38,950 = 7,076,454 parameters		

Figure B.16. DenseNet-121 as it runs in this project, with the tensor shape and parameter count at each stage. The four dense blocks hold 6, 12, 24 and 16 layers. The stem downsamples by four before the first block, and each of the three transitions halves again, giving a total downsampling factor of 32.

The 121 in the name counts the parameterised layers on the main path: one stem convolution, $2 \times 58 = 116$ convolutions across the four dense blocks, three transition convolutions, and the classifier. Keras represents this as 427 layer objects once the normalisations, activations, poolings and concatenations are counted separately, and 427 is the figure the unfreeze logic in B.11.6 operates on.

The spatial arithmetic at 128 pixels is worth following, because one of its consequences reappears in the interpretability work. The stem's stride-2 convolution and stride-2 max pooling take 128 down to 32 before the first dense block. The three transitions then take it to 16, 8 and finally 4. The last convolutional layer therefore produces a 4×4 grid — sixteen spatial cells, each summarising a 32-pixel square of the input. That is the grid Grad-CAM reads, and it is the source of the resolution confound discussed in B.11.17.

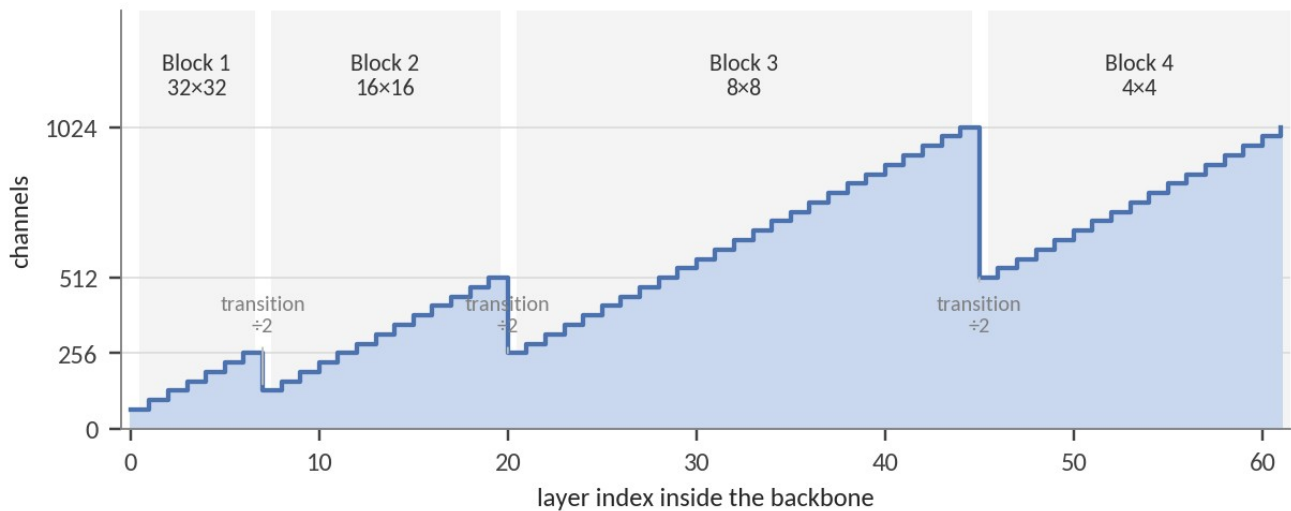


Figure B.17. Channel count through the backbone. Within each block the count climbs by 32 per layer; each transition halves it. The sawtooth is the architecture's defining shape — accumulate, then compress.

Figure B.17 shows the accumulate-and-compress rhythm directly. Block 1 takes 64 channels to 256 over six layers; transition 1 halves that to 128; block 2 takes 128 to 512 over twelve layers; and so on. Block 3 is the largest at 24 layers and carries 2.71 million of the backbone's 7.04 million parameters, which is why it is also where fine-tuning has the most to change.

B.11.5 The model as assembled

The backbone is not used bare. It sits inside a small functional wrapper that carries the preprocessing into the saved artefact:

1	Input (128, 128, 3) — pixels in 0–255, exactly as the tf.data pipeline delivers them
2	augment — RandomFlip, RandomRotation(0.05), RandomZoom(0.05); train-only by construction
3	densenet.preprocess_input — the backbone's own scaling and channel normalisation
4	base_model(x, training=False) — the DenseNet-121 backbone
5	GlobalAveragePooling2D — $4 \times 4 \times 1024 \rightarrow 1024$
6	Dropout
7	Dense(38, activation="softmax")

Table B.4. The assembled model. Steps 2 and 3 are inside the network rather than in the data pipeline, so the saved .keras file cannot be fed wrongly scaled input.

Two details in this stack are not cosmetic. Using `densenet.preprocess_input` rather than a generic `Rescaling(1/255)` matters because each Keras application family expects its own input convention, and feeding ImageNet weights the wrong one degrades the features silently — there is no error, only a worse score. And calling the backbone with `training=False` explicitly, rather than relying on the phase-2 `trainable` flags, keeps every BatchNorm layer in inference mode throughout both phases, which is the mechanism described in B.11.8.

B.11.6 The parameter budget

Where the parameters are

7,076,454 parameters

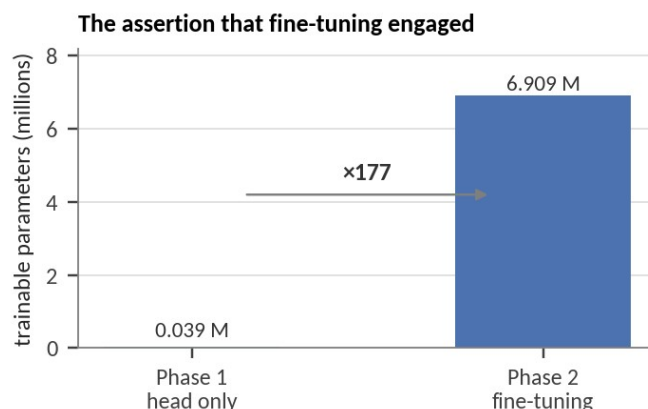
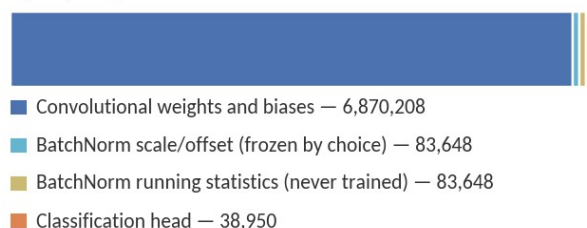


Figure B.18. Left: where the 7,076,454 parameters sit. Right: the trainable count before and after unfreezing — the quantity the phase-2 assertion checks.

The budget decomposes exactly, and the arithmetic is worth setting out because it is what makes the freezing decision auditable rather than asserted:

Component	Parameters	Trainable in phase 1	Trainable in phase 2
Backbone — convolutional weights and biases	6,870,208	no	yes
Backbone — BatchNorm scale and offset	83,648	no	no — frozen by choice
Backbone — BatchNorm running statistics	83,648	no	no — never trainable
Classification head	38,950	yes	yes
Total	7,076,454	38,950	6,909,158

Table B.5. The parameter budget. The two BatchNorm rows are equal by construction — every normalisation layer carries one scale and one offset per channel, both trainable in principle, alongside one running mean and one running variance per channel, never trainable. Freezing the layers moves the first pair into the frozen column, and $7,076,454 - 167,296 = 6,909,158$ is exactly the phase-2 figure the run printed.

That last identity is the reason this table is here. The phase-2 trainable count was not read off a report or copied from a teammate's screenshot; it falls out of the architecture and the freezing decision, and the number the run printed matches it to the digit. Anyone checking this work can verify the freeze actually happened without re-running anything.

B.11.7 The two-phase protocol

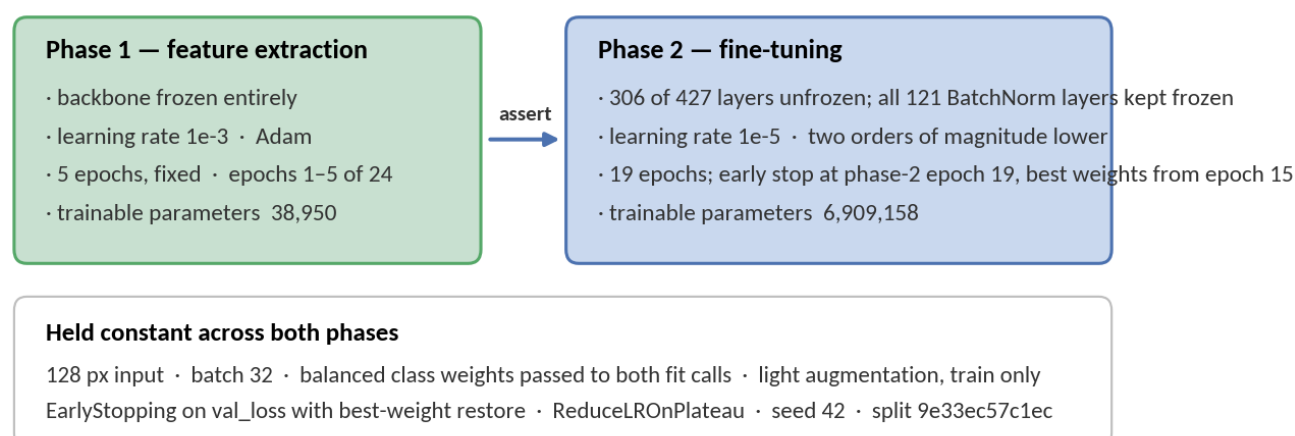


Figure B.19. The training protocol. Phase 1 fits the classification head against fixed ImageNet features; phase 2 refines the backbone at a learning rate two orders of magnitude lower. The assertion between them is a guard, not a formality — it was added after a run in which phase 2 silently trained nothing.

Phase 1 — feature extraction. The backbone is frozen entirely and only the new 38-way classification head is trained, at Adam with a learning rate of 1e-3 for five fixed epochs. The head starts from random weights, so its initial gradients are

large; letting those gradients propagate into a carefully pretrained backbone would damage features that took 1.4 million ImageNet photographs to learn. Freezing first means the head arrives at phase 2 already producing sensible predictions, and the gradients entering the backbone are correspondingly small.

Phase 2 — fine-tuning. The backbone is unfrozen — 306 of the 427 layers — and training continues at 1e-5. The two-order-of-magnitude drop is the operative choice: at 1e-3 the pretrained filters would be overwritten within an epoch, which is functionally training from scratch with an expensive initialisation. At 1e-5 they are nudged towards leaf texture while retaining their general structure. Early stopping on validation loss triggered at epoch 19, and ``restore_best_weights`` returned the network to its epoch-15 state, so the saved checkpoint is the best epoch rather than the last.

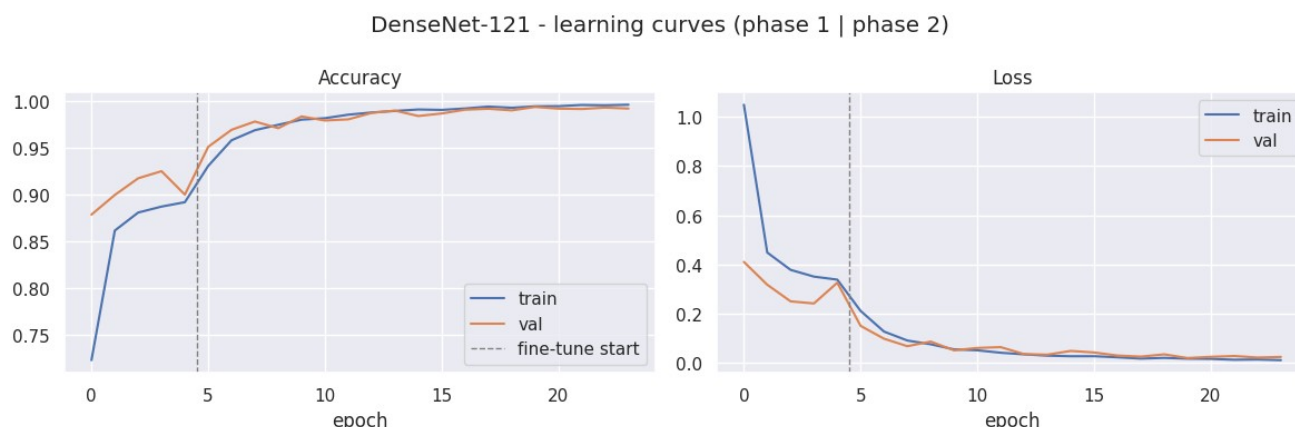


Figure B.20. The final model's own learning curves, reproduced unedited from ``09_transfer_densenet121.ipynb``. The dashed line is the fine-tuning boundary. Compare against the MobileNetV2 curve in Figure B.11 — same plotting code, same two-phase recipe, and no oscillation after the unfreeze. That difference is the BatchNorm freeze.

Four things in this curve are worth reading off directly. **The epoch accounting is visible:** the boundary sits between the fifth and sixth epoch, and the traces run to the twenty-fourth, which is 5 fixed epochs of phase 1 followed by 19 of phase 2 — the figure is what settles an ambiguity the run log alone left open. **Phase 1 plateaus near 0.92 validation accuracy** with the backbone entirely frozen, which is approximately where a fully fine-tuned MobileNetV2 finished (0.932, B.8): a 38-way linear head on untouched ImageNet features already matches a smaller backbone that was allowed to adapt. **Validation accuracy sits above training accuracy for the first eight epochs or so.** That looks alarming and is not: dropout and augmentation are active during training and inactive at validation, so the training metric is measured on a deliberately handicapped network. The gap closes as the model converges, which is the expected pattern. **The unfreeze pays off immediately** — validation accuracy moves from roughly 0.91 to 0.95 in a single epoch and reaches 0.98 by the tenth, then improves slowly to 0.993.

What the curve does *not* show is the more important part. There is no oscillation after the boundary, no spike in validation loss, and no divergence between the two traces — the loss falls smoothly from 0.34 at the boundary to about 0.02 at convergence. Set that against Figure B.11, which is the same protocol with the normalisation layers left free to move: that run shows exactly the instability described in B.11.8. The two figures together are the empirical case for the freeze, and they are the reason this appendix treats it as a finding rather than a convention.

The unfreeze is deep — 306 of 427 layers, essentially everything except the normalisations. This is worth flagging honestly. A shallower unfreeze preserves more of the inherited general structure and adapts less to the laboratory distribution; a deep unfreeze maximises fit to PlantVillage specifically. The deep unfreeze produced the best validation score and is what the reported model uses, but it is also the configuration most specialised to detached leaves on a plain backdrop. Part I's outlook names a shallower re-fine-tune as the natural configuration to test once field imagery exists, and that recommendation follows directly from this choice.

B.11.8 Why the normalisation layers stay frozen

Every one of the 121 BatchNorm layers keeps its inherited ImageNet running statistics throughout both phases. This was the single change that made fine-tuning in this project reliable, and the reasoning deserves to be recorded properly because two earlier runs failed without it.

A BatchNorm layer holds two kinds of state. Its scale and offset are learned parameters. Its running mean and variance are statistics, updated by an exponential moving average over training batches and used unchanged at inference. When

a backbone is unfrozen in the usual way, both start moving at once: the statistics adapt to the new data distribution while the convolutional weights that produce that distribution are themselves being updated. Each is chasing the other. With a batch size of 32 the per-batch estimates are noisy, and the result is loss curves that oscillate or diverge for reasons that have nothing to do with the learning rate.

There is a second, subtler failure. During training a BatchNorm layer normalises using the current batch's statistics; at inference it uses the running averages. If the two have drifted apart — which is exactly what happens when the statistics are still moving — a model can show a healthy training accuracy and a poor validation accuracy that has nothing to do with over-fitting. That pattern was visible in the first MobileNetV2 run and was initially misread as over-fitting.

Freezing removes both problems by fixing the normalisation to a known reference. Mechanically it is done twice over: the layers are excluded from the unfreeze loop, and the backbone is called with `training=False`, which forces inference-mode behaviour regardless of the trainable flags. Either alone would suffice; both together mean a future edit to one cannot silently undo the other.

B.11.9 The assertion that fine-tuning engaged

After the unfreeze and before phase 2 begins, the notebook asserts that the trainable parameter count has risen meaningfully. In this run it rose from 38,950 to 6,909,158, a factor of 177.

The guard exists because of a specific failure. Under Keras 3, setting `base_model.trainable = False` after a per-layer unfreeze loop overrides the layer-level flags, and the ResNet50 run described in B.9 did exactly that. Phase 2 ran for its full epoch budget, printed a learning curve, produced a checkpoint and reported a plausible validation score — while training zero parameters. Nothing about the output distinguished it from a successful fine-tune except the number the assertion now checks.

This is the general shape of the most dangerous class of bug in this kind of work. It does not raise an exception, it does not produce an implausible number, and it is invisible in every artefact except the one nobody thinks to look at. The DenseNet notebook additionally sets `base_model.trainable = True` *before* the per-layer loop rather than after, which avoids the trap at source; the assertion is the belt to that pair of braces.

B.11.10 Callbacks, checkpointing and what was saved

EarlyStopping	monitor val_loss, patience 4, restore_best_weights=True
ReduceLROnPlateau	monitor val_loss, factor 0.5, patience 2, min_lr 1e-7
Save point	immediately after training, before any evaluation cell runs
Saved to	02_models/densenet121.keras, written locally then copied to Drive with byte-size verification
Results	03_results/ — densenet121_results.csv, per-class recall, full classification report
Figures	04_figures/ — row-normalised confusion matrix, learning curves across both phases

The ordering of the save is deliberate and was learned the hard way. Writing the checkpoint before evaluation means a runtime recycle during the prediction pass costs a few minutes rather than seventy-two. The local-then-copy pattern with a size check exists because writes directly to a mounted Drive folder had previously produced zero-byte files without raising.

`restore_best_weights=True` on the early stopper is what makes the phase-2 epoch-15 weights the saved ones. Without it the checkpoint would hold phase-2 epoch 19 — four epochs past the validation-loss minimum — and the reported validation score would be measuring a worse model than the one the curve in B.11.7 identifies.

B.11.11 Compute

Total training time was 72 minutes on a T4. The two phases are very differently priced: with the backbone frozen there is no backward pass through 7 million parameters and an epoch costs under a minute, while phase 2 backpropagates through the whole network and costs several times that. Roughly nine tenths of the 72 minutes belongs to phase 2.

Against the scratch CNN's twenty minutes this is a real cost, but it is paid once and it is not the cost that would matter in deployment. A forward pass through DenseNet-121 at 128 pixels is approximately 0.9 billion multiply-accumulates — estimated by scaling the architecture's published figure of about 2.8 billion at 224 pixels by the square of the resolution ratio — against roughly 0.20 billion computed directly for the scratch CNN. Twenty-seven times the parameters, but only about four and a half times the arithmetic, because DenseNet's parameters sit in deep blocks operating on 8×8 and 4×4 grids where each weight is applied few times, while the scratch CNN's few parameters sit at high resolution where each is applied often.

B.11.12 Validation and the sealed test set

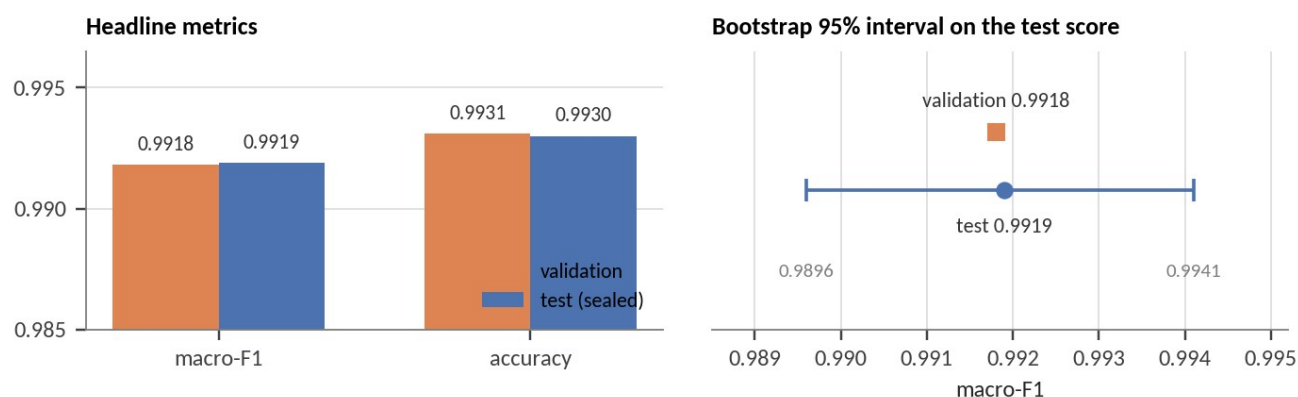


Figure B.21. Left: headline metrics on validation and on the sealed test partition. Right: the bootstrap 95% interval around the test macro-F1, with the validation figure marked for comparison. The interval is ± 0.0023 , narrow because the test set is large.

The test partition had not been read by any model at any point before this evaluation. It was opened once, for this model, and the notebook writes a seal file on completion and refuses to run a second time unless that seal is deliberately removed. The evaluated checkpoint is identified by MD5 hash — ``afe3478450c41d91a6ac1e57a2ada44d`` — so the result is tied to a specific file rather than to a description of one.

	Validation (8,146)	Test (8,146, sealed)
Macro-F1	0.9918	0.9919 95% CI [0.9896, 0.9941]
Accuracy	0.9931	0.9930
Errors	56 (0.69%)	57 (0.70%)
Minimum per-class recall	0.9481	0.9481
Classes at recall 1.000	—	20 of 38
Classes below recall 0.95	—	1 of 38

Table B.6. The final model on both partitions. The confidence interval resamples the 8,146 test predictions with replacement and takes the middle 95% of the resulting macro-F1 distribution.

The gap between validation and test is $+0.0001$ — the model scores fractionally better on data it was never evaluated against than on the data used to select its checkpoint. There is no measurable over-fitting, and the result is credible precisely because the test set was sealed: had it been consulted during development, a gap this small would carry almost no information, because the test set would by then have been part of the selection loop.

The minimum per-class recall is 0.9481 on validation and 0.9481 on test, identical to four decimal places. That coincidence is not luck — it says the hardest class is hard for a structural reason rather than a sampling one, which is the argument developed in B.11.13.

B.11.13 How the sealed evaluation was enforced

A sealed test set is a claim about process, and a claim about process is only as good as whatever makes it hard to break. The evaluation notebook enforces it in code rather than by convention, and the mechanisms are recorded here because the credibility of the $+0.0001$ gap rests on them.

- **The gate runs before any image is read.** The first substantive cell recomputes ``SPLIT_ID`` from the bytes of the three CSVs and halts if it does not read ``9e33ec57c1ec``. If the partition on the machine is not the frozen one, every earlier validation number is incomparable and there is no point opening it — so the notebook stops there, deliberately, before the test set is touched.
- **One prediction call.** The notebook contains exactly one ``model.predict``. Every metric, the bootstrap interval, the per-class recalls and the confusion matrix are computed from that single prediction array. There is no code path that scores a second model or a second configuration.
- **A rehearsal mode.** ``DRY_RUN = True`` runs the entire pipeline against the **validation** split, writes to ``_DRYRUN`` filenames and does not touch the seal. It doubles as a checkpoint check: the validation macro-F1 it prints should

reproduce the figure on record for that model. The rehearsal was run first, the number confirmed, and only then was the flag cleared.

- **A run lock.** On completion the notebook writes `03\_results/final\_test\_evaluation\_SEAL.txt` and refuses to run a second time unless that file is deliberately removed via an explicit `FORCE\_UNSEAL` flag. Re-running would not corrupt the first result, but it would make it possible to quietly report the better of two, which is the thing the seal exists to prevent.
- **The checkpoint is hashed.** The MD5 of the loaded file is printed and recorded alongside the metrics, so the reported score is attached to a specific artefact rather than to a filename that could later hold different weights.

None of this makes the protocol tamper-proof, and it is not meant to. It makes accidental violation difficult and deliberate violation visible in the artefacts, which is the realistic standard for a project of this kind. The order of operations matters more than any single guard: the final-model decision was closed on validation evidence before this notebook was run, so there was no live question that the test result could have been used to settle.

B.11.14 Where the errors are

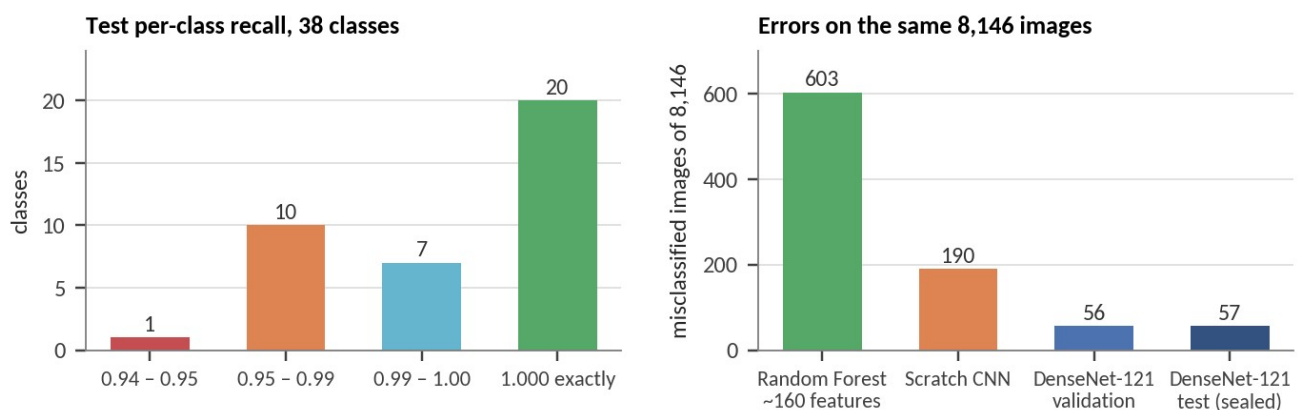


Figure B.22. Left: the distribution of test per-class recall across all 38 classes. Right: misclassified images out of the same 8,146 for three models. The classical figure is derived from its recorded accuracy of 0.926.

Twenty of the thirty-eight classes are recalled perfectly on the test set and only one falls below 0.95. Set against the ladder, the final model makes 56 validation errors where the scratch CNN makes 190 — a reduction of 3.4× — and where the Random Forest makes approximately 603, a reduction of about 10.6×. Both ratios are far outside the ± 0.02 noise band that governs the headline macro-F1 comparison, which is why Part I rests part of its argument on the error structure rather than on the margin alone.

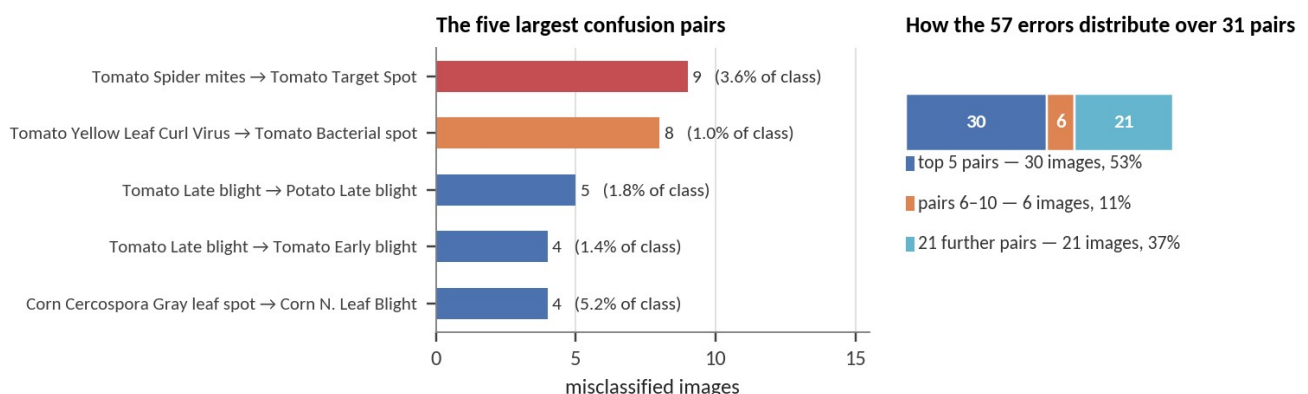


Figure B.23. Left: the five largest confusion pairs on the sealed test set, with each pair's size relative to its true class. Right: how all 57 errors distribute across the 31 distinct pairs. Counts for pairs six to ten are shown in aggregate.

True class → predicted class	Images	Share of class	Share of errors
Tomato Spider mites → Tomato Target Spot	9	3.6%	15.8%
Tomato Yellow Leaf Curl Virus → Tomato Bacterial spot	8	1.0%	14.0%
Tomato Late blight → Potato Late blight	5	1.8%	8.8%
Tomato Late blight → Tomato Early blight	4	1.4%	7.0%
Corn Cercospora Gray leaf spot → Corn Northern Leaf Blight	4	5.2%	7.0%

Table B.7. The five largest confusion pairs. The third and fifth are reciprocal — one image runs the other way in each case — which indicates genuine mutual similarity rather than one class being absorbed into a larger neighbour.

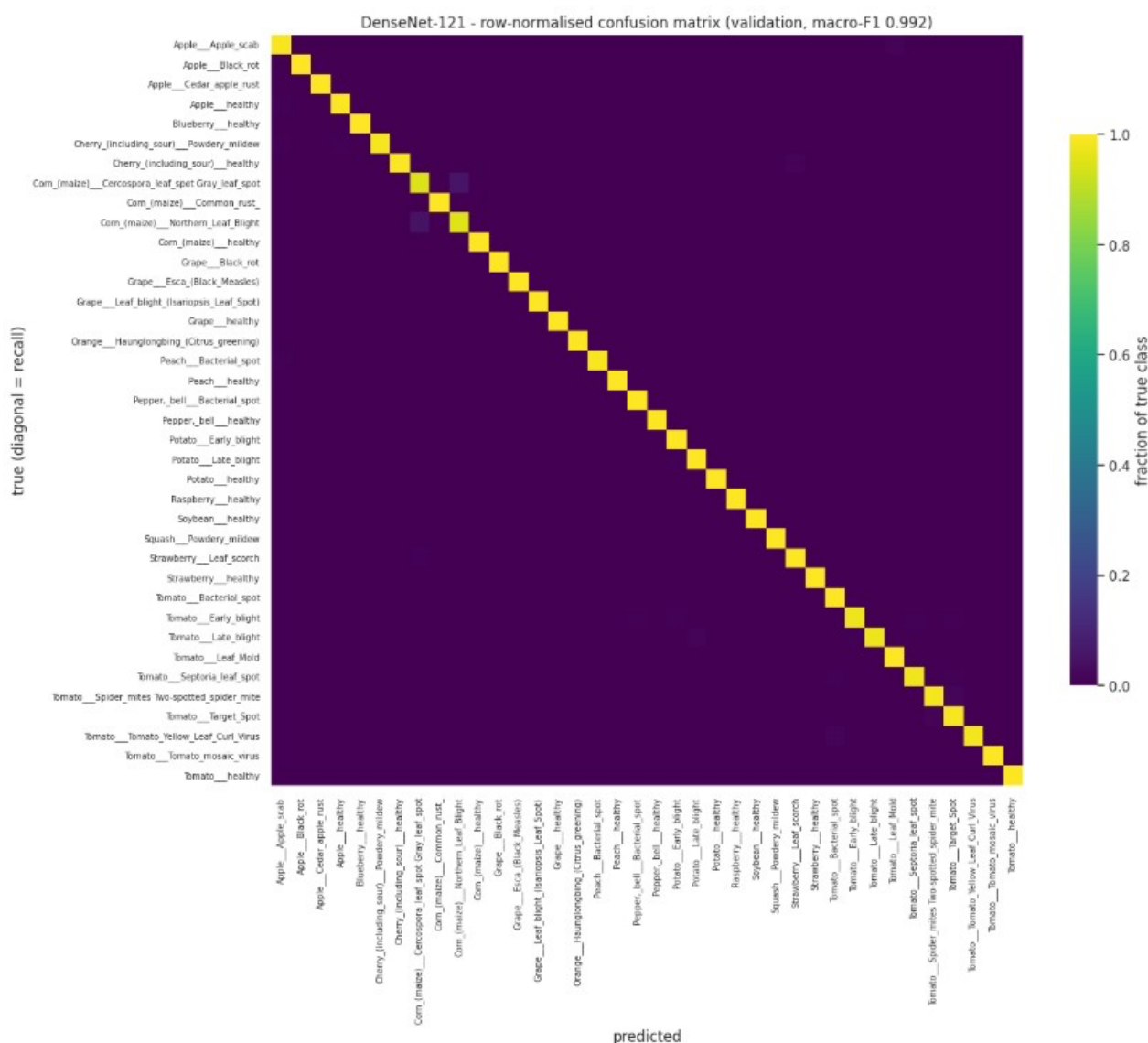


Figure B.24. The final model's row-normalised confusion matrix on the validation set, reproduced from the run notebook; the title carries the 0.992 macro-F1 rounded from 0.9918. The two screenshots the original was captured in have been spliced at the matrix boundary — the cell pitch and column positions are identical in both, so the join is exact. Row and column order are the same sorted class list, so the row labels also name the columns. Best read at screen zoom.

The matrix is worth looking at for what is almost absent from it. The diagonal is uniformly bright, and the only off-diagonal structure visible without zooming is a single block in the Corn rows — Cercospora leaf spot against Northern Leaf Blight, showing on both sides of the diagonal because the confusion runs in both directions. Faint cells appear in the Tomato block and correspond to the pairs in Table B.6. Everything else is empty: on 8,146 validation images, 56 land off the diagonal, and they are concentrated in two crop neighbourhoods rather than scattered across the 38 classes.

Three patterns run through these and each means something different for how the tool would behave in use. **Fine-grained confusion within a crop** — spider mite damage read as Target Spot, Late blight read as Early blight — is the

core difficulty of the task and carries real diagnostic cost, because the treatments differ. **Confusion between crops sharing a pathogen** — Tomato Late blight predicted as Potato Late blight — is the least costly error in the table: *Phytophthora infestans* produces genuinely similar lesions on both hosts, the pathogen is identified correctly, and the host is something the person holding the phone already knows. **The Corn pair** is a stable ceiling. It sets the minimum per-class recall, it is 0.9481 on both partitions, and it appeared in every model built in this project from the Random Forest upward. At 128 pixels the two lesion types are close to indistinguishable, which makes it a resolution hypothesis rather than a modelling failure — and a testable one, by retraining that crop's classes at 256.

What is absent from the table matters as much as what is in it. None of the five largest pairs involves a diseased leaf being classified as healthy, which is the failure mode with the highest practical cost. That absence is a direct consequence of optimising macro-F1, which would not let the rare and difficult classes be averaged away during development.

B.11.15 Grad-CAM — mechanics, and a bug worth recording

Grad-CAM reads the last convolutional layer of the network, computes the gradient of the chosen class's score with respect to each of that layer's feature maps, averages those gradients spatially to obtain one weight per map, and forms a weighted sum of the maps passed through a ReLU. The result is a coarse heat map over the input: warm where the evidence for the prediction came from, cool where it did not. It answers one question — where did the model look — and should not be stretched to answer others. It does not say what would have changed the model's mind, and it is not evidence that the model understands plant pathology.

Implementing it on this model surfaced a silent failure that is worth recording for anyone rebuilding the interpretability notebook. The standard recipe reconstructs a sub-graph by iterating over the model's layers and re-applying them up to the target convolution. Under Keras 3, `'densenet.preprocess_input'` is not a `'Layer'` object — it produces graph operations directly — so the layer iteration skips it entirely. The rebuilt graph then receives raw 0–255 pixels where the backbone expects normalised input. No exception is raised, a heat map is produced, and it is wrong.

The fix is to take the pre-backbone segment from the original graph via the backbone's inbound node rather than reconstructing it layer by layer. Four further defects were caught during dry runs against a synthetic fixture: a ragged results concatenation, a verdict that called a difference significant on a Mann-Whitney p-value while the bootstrap interval spanned zero, a deprecated matplotlib argument, and a fragile re-anchoring guard. The measurements below were produced after all five were fixed.

B.11.16 Turning heat maps into a number

Heat maps are persuasive and easy to over-read, so the question was converted into a measurement. PlantVillage ships a segmented rendering of every image — the same photograph with the background masked out, produced by the dataset's own authors — which supplies a free outline of the leaf. For each image, the share of the heat map's mass falling inside that outline is divided by the share of the frame the leaf occupies. The resulting quantity, called concentration, reads simply: 1.0 means no preference at all, above 1.0 means the model concentrates on the leaf, below 1.0 means it preferentially reads a background containing no disease information whatsoever.

The measurement covers the 7,967 validation images that have a segmented counterpart, 97.8% of the validation set. Leaf area averages 47.4% of the frame, identically for both models, since that is a property of the photographs rather than of the networks. Two things make this a fair probe: the masks were produced by the dataset's authors rather than by us, so they cannot be tuned to flatter either model, and neither network ever saw them — all training used the unsegmented colour images. The instrument is genuinely external to the thing it measures.

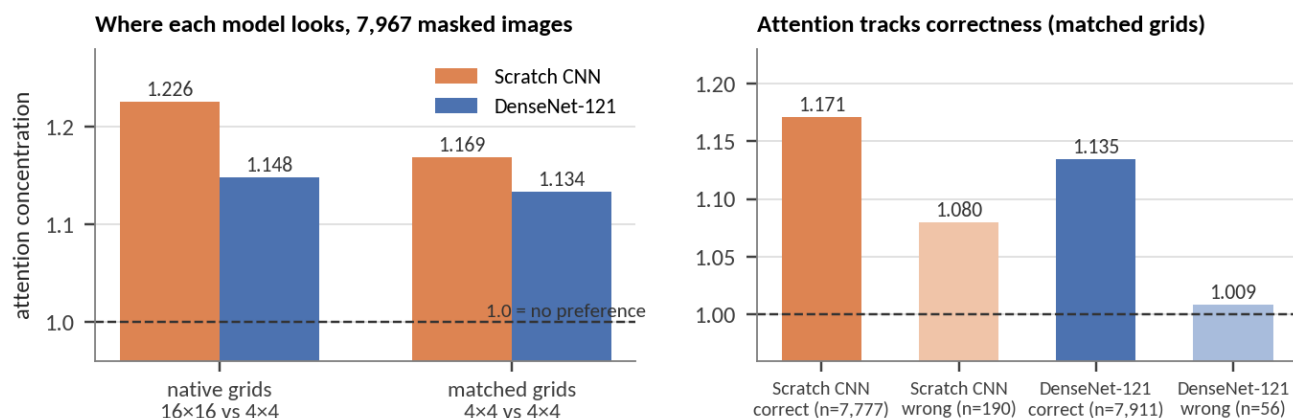


Figure B.25. Left: attention concentration for both models, at their native output grids and recomputed on a common 4×4 grid. Right: concentration split by whether the prediction was correct, at matched resolution.

Model	Attention on leaf	Concentration (native)	Concentration (matched 4×4)
Scratch CNN	56.7%	1.226	1.169
DenseNet-121	53.3%	1.148	1.134
Difference	-3.4 pp	-0.078	-0.034

Table B.8. On-leaf attention across 7,967 masked validation images. Leaf area averages 47.4% of the frame for both models. The matched column recomputes both heat maps on a common 4×4 grid; the 95% intervals on the difference are $[-0.090, -0.066]$ native and $[-0.043, -0.025]$ matched.

Both models prefer the leaf. The scratch CNN places 56.7% of its attention on a leaf occupying 47.4% of the frame; the final model places 53.3% there. Both sit comfortably above 1.0, so neither is reading the backdrop in preference to the plant, and both leave a substantial share of their attention on a plain grey background that carries no disease information at all — a useful corrective to scores in the high nineties.

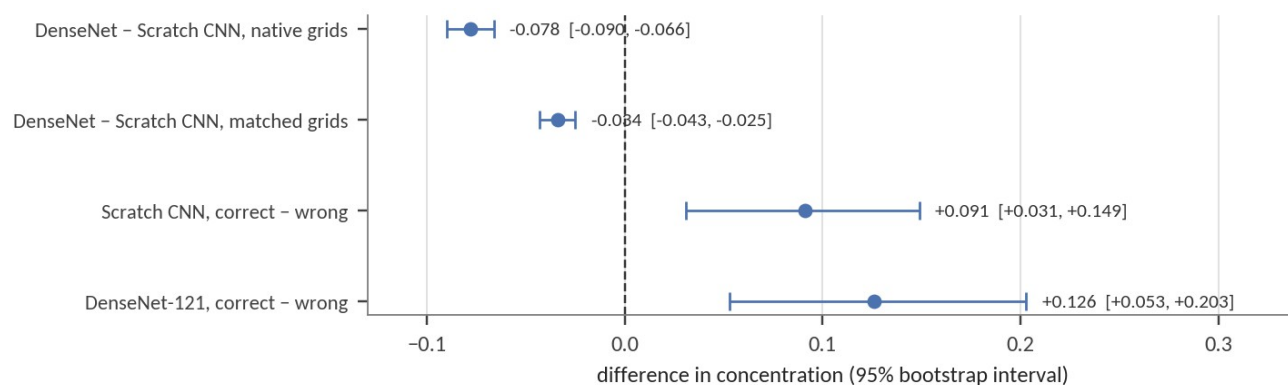


Figure B.26. All four differences with their 95% bootstrap intervals. None crosses zero. The first two say the final model concentrates on the leaf slightly less than the scratch CNN; the last two say both models, and the final model more sharply, look in the wrong place when they are wrong.

The final model's concentration is the lower of the two, and the difference survives the resolution control described next. This was measured as a test of a hypothesis the project had drafted in advance — that an ImageNet-pretrained backbone would depend less on the uniform laboratory backdrop — and the hypothesis was not supported. Part I states this plainly and makes no field-robustness claim for the final model on the strength of its attention. The measurement is reported in full because it is informative, and it plays no part in the selection.

B.11.17 The grid-resolution confound

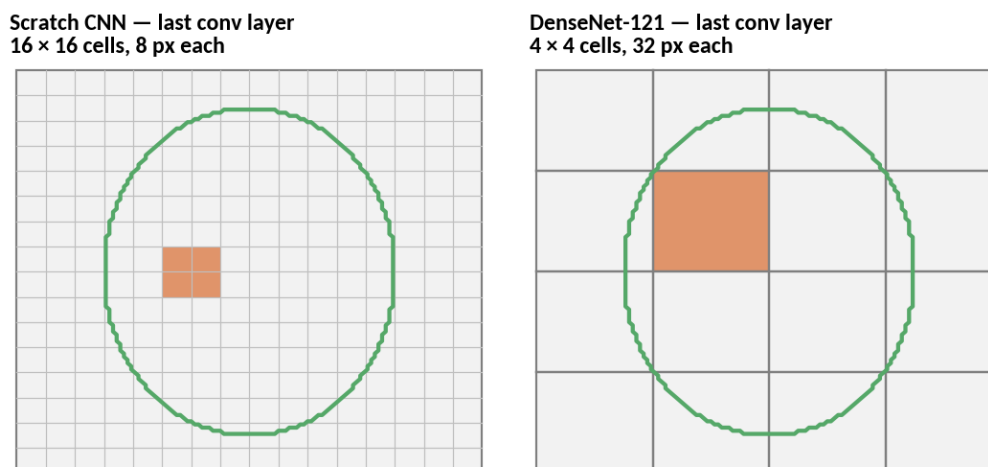


Figure B.27. The two models' Grad-CAM grids drawn to scale over a 128-pixel frame. The scratch CNN's last convolution outputs 16×16 , so one cell is 8 pixels; DenseNet-121's outputs 4×4 , so one cell is 32 pixels. The smallest region either model can attribute evidence to differs by a factor of sixteen in area.

Part of the concentration difference is an artefact of measurement rather than a fact about the models. The scratch CNN's last convolutional layer produces a 16×16 map; DenseNet-121's produces 4×4 , because it downsamples by 32 where the scratch CNN downsamples by 8. A single DenseNet cell covers a 32-pixel square. If a leaf edge runs through that square — and at 47% leaf area many do — the cell's attention is counted as partly off-leaf no matter what the model is actually responding to. A coarser map cannot help but spill across a boundary.

Recomputing both models' heat maps on a common 4×4 grid controls for this. The difference more than halves, from -0.078 to -0.034 , but it does not disappear and its interval still excludes zero. The honest summary is that some of the gap is grid coarseness and some of it is not, and the reported figure is the matched one. A cleaner test would compare heat maps drawn from an intermediate DenseNet block at 8×8 or 16×16 , which Part I lists as an open item.

B.11.18 Attention as a runtime confidence signal

Splitting concentration by outcome produced the most practically valuable result of the interpretability work, and it is the one finding here that survives the caveats intact.

Model	Correct	Wrong	Difference (95% CI)	Mann-Whitney p	Rank-biserial
Scratch CNN	1.171 (n=7,777)	1.080 (n=190)	+0.091 [+0.031, +0.149]	4.4×10^{-6}	+0.195
DenseNet-121	1.135 (n=7,911)	1.009 (n=56)	+0.126 [+0.053, +0.203]	7.1×10^{-4}	+0.262

Table B.9. Concentration split by whether the prediction was correct, both models, at matched 4×4 resolution. Both effects are supported by the bootstrap interval and the rank test, and the effect is larger in the final model.

A concentration of 1.009 is no leaf preference at all. On the images the final model gets wrong, its attention is spread as though the leaf were not there. Misclassification here is therefore not a subtle judgement between two plausible diseases — it coincides, measurably, with the model reading the wrong part of the frame.

That is directly usable, and it is the clearest route from this work to a safer tool. Concentration can be computed at inference time from the model's own gradients, without a ground-truth label and without the segmentation mask — a heat map's spatial spread is measurable on its own. A prediction whose attention is diffuse can be flagged for human review. The effect is stronger in the final model than in the scratch CNN, which Part I records as a fifth practical consideration supporting the choice, and it converts Grad-CAM from a presentation device into a runtime instrument.

B.11.19 What it costs to serve

The model has to run inside a Streamlit application at the end of the project, and the curriculum's requirement is explicit that nothing may be retrained at runtime. Everything below concerns loading a saved checkpoint and calling it, which is a different cost profile from training and a much cheaper one.

Checkpoint	densenet121.keras — a zip archive holding the graph, the weights and the preprocessing
Weight footprint	≈27 MB at float32; roughly 7 MB if post-training quantised to int8
Input contract	a $128 \times 128 \times 3$ array with pixels in 0–255 — no external preprocessing needed
Forward pass	≈0.9 G multiply-accumulates per image (estimated, see B.11.11)
Batching	not required; the application classifies one photograph at a time

The preprocessing being inside the model is what makes this straightforward. A Streamlit page can read an uploaded file, resize it to 128×128 , and pass the raw array straight to `predict` — the rescaling and the ImageNet channel normalisation are layers in the graph, so there is no way for the application to feed the model input it was not trained on. The augmentation layers are in the graph too and are inactive at inference by construction, so they cost a graph node and nothing else.

Inference cost is not a reason to prefer either candidate. DenseNet-121 is roughly four and a half times the arithmetic of the scratch CNN at this input size and both are far below the threshold at which a user would perceive a delay on a single image; on a CPU-only host either returns a prediction in well under a second. The 27 MB checkpoint is larger than the scratch CNN's roughly 1 MB, which matters for a container image and not for anything else. Against that, the argument from simplicity — that a 264-thousand-parameter model is easier to reason about and cheaper to retrain — is real, and Part I records that it was taken seriously and lost on merit.

One deployment consideration does follow from B.11.17 and is worth carrying forward. Computing attention concentration at inference requires a second backward pass to obtain the gradients, roughly doubling the cost of a prediction. For a tool that classifies one photograph at a time on demand, that is affordable, and it buys a confidence signal that needs no ground truth. It would be the first thing to add to the application after the classifier itself works.

B.11.20 Reproducing this model

Everything needed to rebuild the reported checkpoint is fixed and recorded. In order:

- Load the canonical split from `'00_split/'` and confirm the printed fingerprint reads `'9e33ec57c1ec'`. If it does not, stop — nothing downstream is comparable.
- Build the `'tf.data'` pipeline at 128 pixels, batch 32, pixels left in 0–255, validation unshuffled.
- Compute balanced class weights from the training split and confirm the range prints as 0.26 to 9.44.
- Assemble the model as in Table B.6: augmentation, `'densenet.preprocess_input'`, the ImageNet backbone called with `'training=False'`, global average pooling, dropout, a 38-way softmax.
- Phase 1 — backbone frozen, Adam at $1e-3$, five epochs, `'class_weight'` passed.
- Set `'base_model.trainable = True'`, then freeze every BatchNorm layer in a per-layer loop — in that order. Assert the trainable count rises to 6,909,158.
- Phase 2 — Adam at $1e-5$, EarlyStopping on validation loss with patience 4 and `'restore_best_weights=True'`, ReduceLROnPlateau, `'class_weight'` passed again.
- Save to `'02_models/densenet121.keras'` before evaluating anything, and check the file size on Drive.

Expect a validation macro-F1 near 0.993 and a minimum per-class recall near 0.946. More than about 0.01 away from those figures means something has changed, and the split fingerprint is the first thing to check. The seed is 42, but seeding does not make a GPU run deterministic — the ± 0.02 session variation described in A.1 applies here as it does everywhere else in this project.

B.11.21 The selection record

How the final model came to be chosen is part of its technical record, because a score reported on a sealed partition means one thing if the model was picked before that partition was opened and something weaker if it was not.

A decision rule was written down before the interpretability evidence existed. It said, in substance: if the attention measurement separates the two leading candidates materially, prefer the model that attends to the leaf; if it does not separate them, fall back to validation macro-F1. Writing it in advance was the point. Either outcome would then be reportable without the choice looking like an argument constructed after the fact to justify a number already preferred.

The measurement came back against the pretrained model. DenseNet-121 attends to the leaf slightly *less* than the scratch CNN — matched concentration 1.134 against 1.169, a difference of -0.034 with an interval of $[-0.043, -0.025]$ that excludes zero. That is a real difference and it is small: both models sit between 1.13 and 1.17 on a scale where 1.0 means no preference at all, and neither reads the backdrop in preference to the plant. The rule's first clause was

therefore not triggered — the measurement does not separate the candidates materially — and the decision fell to validation macro-F1, where DenseNet-121 leads 0.9918 to 0.973.

Two consequences follow and both are recorded in Part I. A field-robustness argument that an earlier draft had built on architecture — that ImageNet pretraining on 1.4 million varied photographs would make the model less dependent on a uniform laboratory backdrop — was tested here and not supported, and it was withdrawn rather than softened. What replaced it is a different and narrower claim: that a pretrained backbone is the better *starting point* for adaptation to field imagery, because transfer learning pays off most where data is scarce and field data will be scarce even though PlantVillage is not. That claim is about a future fine-tune and is not something this dataset can settle either way, which is why the report states it as a forward-looking argument rather than as a measured result.

The supporting evidence for the choice is the error structure rather than the margin. On the same 8,146 validation images the final model makes 56 errors against the scratch CNN's 190, a 3.4× reduction that sits far outside the ± 0.02 macro-F1 noise band, and the reduction is concentrated in the classes that were hardest — every class the scratch CNN's error analysis had named as difficult is lifted clear of 0.95 recall. A model that had merely got lucky would improve where there was already room to improve.

C. Cross-model ledger

The two tables below gather what is scattered through Section B, so that a reader checking a single figure does not have to find the record it came from.

C.1 Architecture and cost

Model	Representation	Params	Input	Val macro-F1	Compute
Dummy — most frequent	none	0	—	0.005	<1 s CPU
Dummy — stratified	none	0	—	0.026	<1 s CPU
Logistic regression	6 global features	266	full res	0.274	≈1.5 min CPU
Random Forest, 6 features	6 global features	300 trees	full res	0.382	<1 min CPU
Random Forest, ~160 features	colour histograms + GLCM	300 trees	64 px	0.906	≈8 min CPU
Scratch CNN	learned from pixels	263,654	128 px	0.973	≈20 min T4
Tuned CNN	learned from pixels	≈264 K	128 px	0.970	≈95 min T4
Transfer — MobileNetV2	ImageNet, fine-tuned	2,306,662	128 px	0.919	≈38 min T4
Transfer — ResNet50	ImageNet, fine-tuned	≈23.7 M	224 px	0.978	≈65 min T4
Transfer — EfficientNetB0	ImageNet, fine-tuned	≈4.15 M	224 px	0.982	≈60 min T4
DenseNet-121	ImageNet, fine-tuned	7,076,454	128 px	0.9918	72 min T4

Table C.1. All eleven models. The logistic regression's parameter count is 38 classes × (6 coefficients + 1 intercept). Forest sizes are given as tree counts, since a forest has no parameter count comparable to a network's. Compute figures include feature extraction for the classical models.

C.2 Provenance

Model	Notebook	Run by	Checkpoint status
Dummies, LogReg, scratch CNN	Step3_1_PlantVillage_Baseline_Models	Christoph	baseline_cnn.keras on Drive
Random Forest, 6 features	Step3_1_PlantVillage_RandomForest_Global6	Christoph	random_forest_global6.joblib
Random Forest, ~160 features	Step3_1_PlantVillage_RandomForest	Christoph	random_forest.joblib
Tuned CNN	Step3_2_PlantVillage_CNN_Tuning	Christoph	tuned_cnn.keras
MobileNetV2	Step3_2_PlantVillage_Transfer_Learning	Christoph	transfer_mobilenetv2.keras; corrected rerun not saved
ResNet50	Step3_2_PlantVillage_ResNet50 (rebuilt)	Scheima	notebook in chat history only
EfficientNetB0	transfer notebook, 224 px	Adrian	notebook in chat history only
DenseNet-121	09_transfer_densenet121	Christoph, on Alexander's recipe	densenet121.keras, MD5 afe3478...a44d
Grad-CAM and attention	11_interpretability_gradcam	Christoph	figures and CSVs on Drive
Sealed test evaluation	12_final_test_evaluation	Christoph	sealed — final_test_evaluation_SEAL.txt written

Table C.2. Where each result came from. Some models were trained across several cloud sessions and a small number of early checkpoints and notebooks were lost to runtime recycles; where a surviving artefact and an earlier document disagree, the surviving artefact is the figure of record.